嵌入式软件代码缺陷与安全漏洞检测系统

调　研　与　技　术　方　案

项　目　名　称：**嵌入式软件代码缺陷与安全漏洞检测系统**

文　档　性　质：**开题调研报告（含技术方案）**

技　术　路　线：**静动双通道骨架 · 多智能体协同 · 工具验证约束**

行　业　场　景：**汽车电子 · 工业控制（兼顾航空航天/军工）**

文　档　版　本：**V1.2（补充调研版）**

完　成　日　期：**2026 年 10 月**

摘　要

嵌入式软件部署于资源受限、强实时、硬件强耦合的封闭运行环境，其代码缺陷直接关联功能安全（Functional Safety）与信息安全（Cybersecurity）。本报告面向汽车电子与工业控制行业场景，围绕嵌入式 C/C++ 代码缺陷与安全漏洞检测系统开展开题调研，完成三项工作。其一，梳理法律法规、国家标准、行业标准与编码规范对代码级检测活动的强制要求，并以工业控制系统漏洞增长、汽车软件召回占比等可核查数据论证立项必要性。其二，按通用软件分析工具、嵌入式与 C/C++ 专项工具、形式化方法与符号执行、学术里程碑四类，对三十余个代表性工具与项目进行系统综述，并补充调研固件动态分析（Firm-AFL、P2IM、HALucinator、μAFL）与 LLM 增强模糊测试（ChatAFL、Fuzz4All、OSS-Fuzz harness 合成）的相邻工作，逐项核实年份、出处与具体局限，归纳通用工具在嵌入式场景水土不服的八项原因，给出横向对比总表，并对大模型加嵌入式检测的参考项目逐项核实（其中一项未能核实到公开来源，已显式标注）。其三，凝练 P1 至 P7 七个亟需解决的问题，提出静态与动态双通道为骨架、大语言模型（Large Language Model, LLM）多智能体协同为中枢、工具验证与证据链为约束的混合式五层技术架构：静态通道承担规则检查、抽象解释、符号执行与污点分析，动态通道承担覆盖率引导模糊测试、插桩检测、仿真执行与运行时监控；六类 LLM 智能体（规划、静态研判、动态测试、运行时监控、修复、报告）以状态机方式协同，但任何结论须经工具验证与证据回链方可采纳，LLM 不作最终裁决。报告给出编译环境复现、硬件语义与并发建模、规则引擎、动态分析引擎、运行时监控、多智能体协同、结果聚合与可证明性输出等关键模块的可工程化设计，并规划三类评测数据集（含自建嵌入式漏洞数据集）、七配置消融评测方案与分季度实施路线。文中开源项目星标数均为截至 2026 年 10 月的快照数据。

关键词：嵌入式软件；静态分析；动态分析；模糊测试；运行时监控；多智能体；大语言模型；MISRA

目　录

摘　要.....

摘　要.....**2**

第一章　绪论.....**2**

1.1 项目背景：政策与标准引领.....3

1.1.1 法律法规与国家标准.....3

1.1.2 行业标准与编码规范.....3

1.1.3 产业规划与“为什么是现在”的量化证据.....4

1.2 嵌入式软件与通用软件的本质差异.....4

1.3 嵌入式编程语言全景.....6

1.4 系统定位：这套系统到底干什么、能做什么.....7

1.5 研究意义、目标与技术挑战概览.....7

第二章　相关工作综述.....**8**

2.1 分类框架.....8

2.2 A 类：通用软件分析工具与引擎.....8

2.2.1 Coverity（2002）.....9

2.2.2 CodeQL（2019 年开放）.....9

2.2.3 Semgrep（2020）.....10

2.2.4 Infer（2015）.....11

2.2.5 SonarQube（2008）.....12

2.2.6 Clang Static Analyzer（随 LLVM 演进）.....12
2.2.7 PVS-Studio（2008）.....13
2.2.8 Cppcheck（2009）.....14
2.2.9 TscanCode（2015/2016）.....15
2.2.10 AFL++（2019）与 OSS-Fuzz（2016）.....15
2.2.11 固件动态分析研究：Firm-AFL（2019）、P2IM（2020）、HALucinator（2020）、μAFL（2022）、Karonte（2020）.....16
2.2.12 预训练代码模型：CodeBERT（2020）→ GraphCodeBERT（2021）→ UniXcoder（2022）.....17
2.2.13 LLM 检测与 LLM 模糊测试方案：GPTScan（2024）、LLM4Vuln（2024）、ChatAFL（2024）、Fuzz4All（2024）与 OSS-Fuzz harness 合成（2024）.....18
2.3 B 类：嵌入式/C-C++ 专项工具.....19
2.3.1 Astrée（2009 年商用）.....19
2.3.2 Polyspace（2007 年并入 MathWorks）.....19
2.3.3 Frama-C（2008）.....20
2.3.4 CodeSonar（2003）.....21
2.3.5 Klocwork（2001）.....22
2.3.6 Helix QAC（1986/2018）.....22
2.3.7 LDRA（1975）.....23
2.3.8 Parasoft C/C++test（1999）.....24

2.3.9 VectorCAST（1990）.....25
2.3.10 ESBMC（2011）.....25
2.4 C 类：形式化方法与符号执行.....26
2.4.1 CBMC（2004）.....26
2.4.2 KLEE（2008）.....27
2.4.3 CPAchecker（2011）.....28
2.4.4 IKOS（2015）.....29
2.4.5 Goblint（2007）.....29
2.4.6 SV-COMP（2012）.....30
2.5 D 类：学术里程碑.....31
2.5.1 规则挖掘与属性验证时代（2001—2005）：Metal、ESP、MOPS、Saturn31
2.5.2 深度学习时代（2018—2021）：VulDeePecker、Devign、SySeVR、D2A32
2.5.3 大模型与严格基准时代（2024—2025）：PrimeVul、VulBench.....32
2.6 为什么通用工具在嵌入式场景水土不服.....33
2.7 大模型+嵌入式检测的参考项目.....34
2.7.1 LCMHalMCP[未检索到].....34
2.7.2 embedded-ai-workspace（已核实）.....35
2.7.3 中科院软件所嵌入式漏洞挖掘技术（非公开）.....35
2.7.4 通用对照基线：open-code-review（2026）.....36
2.8 横向对比总表与解读.....36

第三章　问题定义与技术方案.....37

3.1 急需解决的问题.....37

3.2 总体技术路线.....38

3.3 关键模块的具体实现.....39

3.3.1 编译命令与构建环境自动复现（对应 P1）.....39

3.3.2 硬件语义与并发建模（对应 P2）.....41

3.3.3 MISRA/GJB/CERT 规则引擎实现（对应 P3）.....42

3.3.4 抽象解释与符号执行的嵌入式适配（对应 P4）.....44

3.3.5 动态分析引擎：模糊测试与仿真执行（对应 P7）.....44

3.3.6 运行时监控子系统（对应 P7）.....46

3.3.7 多智能体协同框架（对应 P5、P7）.....48

3.3.8 LLM 研判与修复机制（智能体内部机制，对应 P5）.....50

3.3.9 增量分析与大规模工程性能优化.....52

3.3.10 结果聚合、可证明性输出与报告生成（呼应 P3、P4）.....52

3.4 数据集.....53

3.4.1 合成/标准数据集.....53

3.4.2 真实世界数据集.....53

3.4.3 自建嵌入式数据集（创新点，单独详写）.....53

3.4.4 数据集使用策略.....55

3.5 评测方案与指标.....55

3.5.1 对比基线.....55

3.5.2 指标定义.....55

3.5.3 消融实验设计.....56

3.5.4 预期结果表（留空待填）.....56

3.6 实施路线与阶段计划.....56

参考文献.....57

注：本目录由域代码生成。编辑文档后，请在目录上单击右键选择“更新域”刷新页码。

第一章　绪论

1.1 项目背景：政策与标准引领

嵌入式软件的代码级检测并非单纯的技术选型问题，而是由法律法规、国家标准、行业标准和编码规范共同构成的合规要求体系。本节按效力层级逐项列明可核查的文件编号与年份，并以量化数据说明立项的时间窗口。

1.1.1 法律法规与国家标准

（1）法律法规。《中华人民共和国网络安全法》于 2016 年通过、2017 年 6 月 1 日施行，2025 年 10 月完成首次修正并于 2026 年 1 月 1 日施行，进一步压实网络运营者的漏洞处置责任[1]；《关键信息基础设施安全保护条例》（国务院令第 745 号）自 2021 年 9 月 1 日施行，将能源、交通、水利、工业制造等行业的工控系统纳入重点保护范围并明确检测义务[2]；《中华人民共和国数据安全法》自 2021 年 9 月 1 日施行[3]。三部法律法规共同构成嵌入式软件（尤其关基场景）开展安全检测的法定依据。

（2）国家标准。GB/T 22239-2019《信息安全技术 网络安全等级保护基本要求》（等保 2.0）于 2019 年发布、同年 12 月实施，将安全要求从网络层延伸至应用与数据层[4]；GB/T 38674-2020《信息安全技术 应用软件安全编程指南》于 2020 年发布，给出覆盖 C/C++ 等语言的安全编程与检测基线[5]；GB/T 30279-2020《信息安全技术 网络安全漏洞分类分级指南》为漏洞分级提供国家口径[6]。功能安全侧，GB/T 34590-2022《道路车辆 功能安全》等同采用 ISO 26262:2018 并于 2023 年 7 月实施[7]，GB/T 20438-2017 系列等同采用 IEC 61508:2010[8]。

1.1.2 行业标准与编码规范

（1）汽车电子（本系统默认行业之一）。ISO 26262 系列对软件单元验证规定了随汽车安全完整性等级（Automotive Safety Integrity Level, ASIL）递增的方法强度：静态分析在 ASIL D 为强推荐项，结构覆盖率最高要求达到修正条件/判定覆盖（Modified Condition/Decision Coverage, MC/DC）[7]。网络安全侧，ISO/SAE 21434:2021 将网络安全

工程贯穿车辆全生命周期[13]；联合国法规 UNECE R155 自 2021 年生效、2022 年 7 月起对新车型强制[14]；欧盟《网络弹性法案》（Cyber Resilience Act, Regulation (EU) 2024/2847）已于 2024 年 12 月生效，漏洞报告义务自 2026 年 9 月 11 日起适用[15]。

（2）工业控制（本系统默认行业之二）。IEC 61508（GB/T 20438-2017）对软件在较高安全完整性等级要求使用编码标准子集与静态分析[8]。轨道交通领域，EN 50128 已被 EN 50716:2023 取代，新标准统一了轨交软件的开发与验证要求[16]。

（3）航空航天与军工（扩展行业）。DO-178C 按设计保证等级（Design Assurance Level, DAL）要求结构覆盖率，A 级强制 MC/DC，且明确规定静态分析不能替代动态测试；GJB 8114-2013《C/C++ 语言编程安全子集》规定 124 条强制性检查规则，GJB 5369-2005《航天型号软件 C 语言安全子集》规定 138 条规则，两者构成国内军品嵌入式软件静态检测的事实基线（军用标准不公开发行，依据标准编号引用）。

（4）编码规范。MISRA（Motor Industry Software Reliability Association）C:2012 于 2013 年发布并经四次修订（AMD1 2016、AMD2 2020、AMD3 2022、AMD4 2023），其整合版 MISRA C:2023 已发布，MISRA C:2025 于 2025 年 3 月发布[17]；美国卡内基梅隆大学软件工程研究所（SEI）发布的 CERT C 编码标准提供 98 条安全编码规则并与 CWE 建立官方映射[18]；MITRE 组织每年发布的通用缺陷枚举（Common Weakness Enumeration, CWE）Top 25 榜单（2025 版基于约 39,080 条 CVE）是漏洞类型优先级排序的权威参考[19]。

1.1.3 产业规划与“为什么是现在”的量化证据

产业规划方面，工信部《“十四五”软件和信息技术服务业发展规划》（工信部规〔2021〕180 号）明确将工业软件与嵌入式软件列为补短板重点[9]；国务院《新能源汽车产业发展规划（2021—2035 年）》（国办发〔2020〕39 号）推动整车电子电气架构软件化[10]；《工业互联网创新发展行动计划（2021—2023 年）》（工信部信管〔2020〕197 号）与《“十四五”国家信息化规划》对工业互联网设备与关基系统安全提出要求[11][12]。

量化证据方面，本报告仅引用可核查数据：其一，据 SynSaber 对美国网络安全和基础设施安全局（CISA）工业控制系统（Industrial Control System, ICS）安全通告的统计，2022 年 ICS 通告共揭示 1,342 个 ICS 漏洞，2020 至 2022 年漏洞数量增长约 144%[20]；其二，Microsoft 2021 年调查显示 80% 的受访企业过去两年内遭遇过至少一次固件攻击[21]；其三，Upstream《2025 全球汽车网络安全报告》统计 2024 年公开报道的汽车网络安全事件达 409 起（2023 年为 295 起），其中可大规模影响车辆群的事件占比升至 19%[22]；其四，2024 年美国车辆召回约 2,890 万辆，其中约 46% 涉及软件问题（2023 年该比例为 14.2%，不同统计口径存在差异，见来源说明）[23]；其五，CISQ 估算 2022 年美国软件劣质成本约 2.41 万亿美元[24]。

上述合规压力与安全态势共同界定了立项窗口：漏洞增长与软件化趋势使检测需求刚性化，而现有工具在嵌入式场景的能力缺口（详见第二章）尚未被填补，**此时构建面向嵌入式的代码缺陷与安全漏洞检测系统具备明确的必要性与时效性。**

1.2 嵌入式软件与通用软件的本质差异

通用代码检测工具的默认假设（类 UNIX 桌面环境、完整标准库、操作系统进程隔离源码可直接编译）在嵌入式场景大面积失效。本节将差异归纳为七个维度并逐项说明其对检测技术的影响，这是全篇“为什么要单独做嵌入式检测”的立论根基。

（1）资源受限。目标平台内存以千字节计、无虚拟内存与内存管理单元（Memory Management Unit, MMU），堆分配常被禁用。检测影响：越界写不会触发段错误而是静默破坏相邻任务内存，缺陷暴露点远离触发点；运行时检测工具（如 AddressSanitizer）因依赖页保护机制在裸机平台不可用。

（2）强实时性。任务有截止期约束，需要论证最坏情况执行时间（Worst-Case Execution Time, WCET）。检测影响：代码模式（动态分配、递归、无界循环）本身成为缺陷；时序缺陷无法靠输入遍历发现。

（3）硬件强耦合。代码经内存映射输入输出（Memory-Mapped I/O, MMIO）直接读写外设寄存器，行为依赖芯片手册。检测影响：分析器必须理解“对地址 0x40021000 的写入”的语义，否则大量告警不可判定（详见 2.6）。

（4）交叉编译。宿主机与目标机指令集、字长、对齐不同，构建由交叉工具链完成且配置分散。检测影响：通用工具依赖的编译数据库在嵌入式工程中经常缺失或不可复现（详见 P1）。

（5）裸机与实时操作系统（Real-Time Operating System, RTOS）环境。并发单位是中断服务程序（Interrupt Service Routine, ISR）与 RTOS 任务而非线程，调度语义由 RTOS API 定义。检测影响：通用数据竞争模型基于线程共享内存假设，对 ISR 与任务间竞争普遍漏报（详见 P2）。

（6）长生命周期、难打补丁。设备服役十年以上，现场升级成本高甚至物理不可达。检测影响：缺陷必须在出厂前拦截，对漏报的容忍度远低于通用软件。

（7）失效代价高。汽车、工业场景的失效直接关联人身与生产安全，受 1.1 节所述标准强制约束。检测影响：检测结果需满足可追溯与可证明性要求，单纯“报告式”输出不满足合规证据链要求。

表 1-1 嵌入式软件与通用软件的本质差异及其检测影响

差异维度	嵌入式场景特征	对检测技术的影响
资源受限	内存千字节级、无 MMU、常禁堆	越界写静默破坏内存；ASan 等运行时工具不可用
强实时性	任务截止期、WCET 约束	动态分配/递归/无界循环成为缺陷；时序缺陷难遍历
硬件强耦合	MMIO 寄存器直接读写、依赖芯片手册	寄存器访问语义需显式建模，否则告警不可判定
交叉编译	交叉工具链、构建配置分散	编译数据库缺失，通用工具导入失败
裸机/RTOS	并发单位为 ISR 与任务，API 定义调度语义	线程模型的数据竞争检测漏报 ISR 竞争
长生命周期	服役十年以上、现场升级困难	缺陷须出厂前拦截，漏报容忍度极低
失效代价高	关联人身与生产安全、受标准强制	检测结果须可追溯、可证明，满足合规证据链

1.3 嵌入式编程语言全景

嵌入式软件并非只有 C/C++。按行业分布与工程实践，与代码检测相关的语言可分为九类，本系统按“C/C++ 为主、兼顾汇编与 Rust 趋势”的口径划定范围，其余语言作为兼容性预留。

（1）C 语言。嵌入式绝对主力，覆盖裸机、RTOS 与嵌入式 Linux 内核态。检测难点集中在前述七个维度，尤其指针运算、隐式类型转换与未定义行为（Undefined Behavior, UB）。

（2）C++ 语言。用于复杂车载域控与工控上位机，子集受 AUTOSAR C++14 与 MISRA C++:2023 约束。检测难点为对象生命周期、异常与模板实例化爆炸。

（3）汇编（ARM/Thumb、RISC-V、8051、PowerPC）。启动代码、中断向量表与关键时序片段仍以汇编实现。检测难点为无中间表示可依赖，需反汇编级或模式级检查（对齐、原子性、寄存器约定）。

（4）Rust。以所有权与借用检查在语言层消除数据竞争与释放后使用（Use-After-Free, UAF），实时执行器框架 RTIC 与 Rust-for-Linux 推动其进入嵌入式。检测重点转移为 unsafe 块边界与 FFI 交互。本系统将其列为趋势跟踪项。

（5）IEC 61131-3 工控语言（ST/LD/FBD）。可编程逻辑控制器（PLC）编程标准语言，编译后多运行于专有 runtime。检测以规则检查与离线仿真为主，作为兼容预留。

（6）MATLAB/Simulink 生成代码。模型驱动开发在汽车控制策略中普遍使用，其生成 C 代码（Embedded Coder）有厂商保证，但手写接口与集成代码仍需检测，且生成代码风格对通用规则引擎不友好。

（7）MicroPython/Lua 脚本层。物联网网关与原型场景常用，类型动态导致静态保证弱，以运行时沙箱与字节码检查为主，列为预留。

（8）Java（车载信息娱乐与 Android Automotive）。属应用层，通用 JVM 生态工具成熟，不在本系统核心范围。

（9）Verilog/VHDL。硬件描述语言属芯片侧验证范畴（仿真与形式验证生态独立），明确排除在本系统范围外。

表 1-2 嵌入式编程语言全景与本系统范围界定

语言	典型场景	检测要点	本系统范围
C	裸机/RTOS/内核态全场景	指针与 UB、ISR 并发、寄存器访问、MISRA/GJB 合规	核心支持
C++	车载域控、工控上位机	生命周期、异常与模板、AUTOSAR/MISRA C++ 子集	核心支持
汇编	启动代码、向量表、关键时序	对齐与原子性、寄存器约定、反汇编级检查	选择性支持
Rust	RTIC、Rust-for-Linux（新兴）	unsafe 边界、FFI 交互、借用检查互补	趋势跟踪
IEC 61131-3	PLC（ST/LD/FBD）	规则检查、离线仿真	兼容预留
Simulink 生成代码	汽车控制策略	手写接口与集成层、生成代码风格适配	接口层支持
MicroPython/Lua	网关与原型脚本层	运行时沙箱、字节码检查	预留
Java（IVI）	车载信息娱乐	通用 JVM 工具生态	范围外
Verilog/VHDL	硬件逻辑	仿真与硬件形式验证	范围外

1.4 系统定位：这套系统到底干什么、能做什么

本系统定位为面向嵌入式 C/C++ 源码的白盒检测平台（有源码为主），采用“静态与动态双通道为骨架、LLM 多智能体协同为中枢”的技术路线，按七层能力组织输出；二进制/固件侧能力作为无源码场景的扩展预留。每一层能力对应明确的缺陷示例，避免“通用质量提升”式的空泛表述。

表 1-3 系统能力分层与典型缺陷示例

能力层	检测内容	典型缺陷示例
规范符合性检查	MISRA C:2012/2023、CERT C、GJB 8114/5369 规则逐条检查与合规记录	隐式整型收窄（MISRA 10.x 族）；使用已废弃函数（GJB 禁用库函数）
代码缺陷与运行时错误	空指针解引用、数组越界、UAF、整数溢出、除零、未初始化、死代码	计数器回绕导致缓冲区索引为负；错误分支漏初始化状态变量

能力层	检测内容	典型缺陷示例
安全漏洞检测	污点分析（Taint Analysis）、CWE 映射、CVE 关联	外部报文字段未校验长度即 memcpy（CWE-787 越界写）；printf 用户可控格式串（CWE-134）
并发与硬件语义缺陷	中断与主循环竞争、重入、volatile 误用、MMIO 访问合法性	ISR 与任务共享多字节变量未加临界区导致数据撕裂；对只读状态寄存器执行写操作
动态触发与运行时监控	覆盖率引导模糊测试、插桩检测、仿真执行、断言/不变量运行时监测、时序观测	边界输入实际触发缓冲区溢出（附 PoC 与轨迹）；运行时捕获 ISR 与任务共享数据撕裂现场；中断延迟超限
二进制/固件侧（扩展）	反汇编、固件模糊测试（Fuzzing）、无源码场景	闭源第三方库中命令注入；固件内硬编码凭证
供应链合规	软件物料清单（Software Bill of Materials, SBOM）、开源成分与许可证	引入带已知 CVE 的旧版 lwIP；GPL 组件链接进闭源固件

1.5 研究意义、目标与技术挑战概览

研究意义。其一，合规意义：为 ISO 26262/IEC 61508/GJB 体系提供可追溯的代码级检测证据链，降低认证人工成本；其二，工程意义：以 LLM 智能体承接告警分诊与解释，缓解静态分析误报消耗专家时间这一产能瓶颈（微软对其 C/C++/C# 代码库的实证研究显示告警处置行为直接影响工具信任[75]）；其三，方法论意义：探索“静动双通道精确分析为骨架、多智能体为协同中枢、工具反馈为约束”的混合式路线，为 LLM 应用于安全关键软件提供可复现的评测证据。

研究目标。（1）在自建嵌入式数据集与 Toyota ITC 基准上，F1 相较单一基线工具（Cppcheck、Clang Static Analyzer）提升不低于 20%；（2）LLM 智能体的误报过滤使人工审查工作量下降不低于 30%，且漏判率不高于基线；（3）对 MISRA C:2012 与 GJB 8114 规则集形成自动检查与合规记录输出；（4）形成可复现的构建环境复现工具链，使第三章选定的目标仓库零人工干预完成分析接入；（5）动态通道对至少三个目标固件产出真实触发的缺陷证据（PoC 与轨迹），并给出静态/动态检出互补率实测数据。

技术挑战概览。挑战一：交叉编译环境与构建语义的自动复现（对应 P1）；挑战二：ISR 并发与硬件寄存器语义的形式化建模（对应 P2）；挑战三：规范检查与漏洞检测两套

体系的统一表示与聚合（对应 P3）；挑战四：误报与漏报的平衡及结论可证明性（对应 P4）；挑战五：LLM 幻觉抑制与证据回链（对应 P5）；挑战六：贴近真实嵌入式缺陷分布的评测数据缺失（对应 P6）；挑战七：动态检测在资源受限平台的落地——激励构造、观测开销与仿真保真度（对应 P7）。七个问题在第三章逐一展开并给出解法。

第二章　相关工作综述

2.1 分类框架

为避免流水账式罗列，本报告按“技术谱系与目标场景”将现有工作划分为四类：A 类为通用软件分析工具与引擎（跨语言、面向通用软件生态，含模糊测试基础设施与固件动态分析研究、预训练代码模型与 LLM 检测方案）；B 类为嵌入式/C-C++ 专项工具（以安全关键行业为主战场，多具备标准合规与工具鉴定能力）；C 类为形式化方法与符号执行（以数学证明或路径完备探索为目标）；D 类为学术里程碑（按规则挖掘、深度学习、大模型三代演进）。四类共同勾勒出技术脉络：从 2000 年代初的编译器扩展规则挖掘，到抽象解释与模型检查的精确化，再到深度学习与大模型两次范式转移；静态之外的动态分支（模糊测试、固件仿真、符号执行测试生成）与静态分支并行发展，两者互补关系是第三章双通道设计的直接依据。2.2 至 2.5 逐类调研，2.6 归纳通用工具的失效原因，2.7 盘点大模型加嵌入式的参考项目；每个项目统一按“出处、技术核心、解决问题、实测效果、局限性”五要素撰写，局限性将直接成为第三章问题定义的论据。

证据纪律说明：本报告所有项目的年份、出处均于 2026 年 10 月经联网核实，核实不到之处以[待核实]或[未检索到]显式标注；星标数、性能数字等动态数据标注其统计口径。

2.2 A 类：通用软件分析工具与引擎

2.2.1 Coverity（2002）

出处：斯坦福大学技术衍生公司，2002 年创立；2014 年被 Synopsys 以 3.75 亿美元收购，2024 年随其软件完整性集团出售并更名归属 Black Duck[25]。

技术核心：全程序过程间数据流/控制流缺陷分析（Checkers 体系），与早期 Metal 研究一脉相承[25]。

解决问题：企业级 C/C++/Java 等大型代码库的静态应用安全测试（Static Application Security Testing, SAST）。

实测效果/落地：Coverity Scan 免费服务开源社区，覆盖 3,900 余个项目、累计分析数十亿行代码[25]。

局限性：分析依赖完整构建捕获（build capture），嵌入式交叉工程经常无法产出可用的构建模型（对应 P1）；规则定制灵活性低于 CodeQL/Semgrep 的查询式方案。

2.2.2 CodeQL（2019 年开放）

出处：查询语言 QL 源于 2006 年牛津大学 Semmle 公司，GitHub 2019 年收购后于同年 11 月开放研究与开源使用[26]。

技术核心：将代码编译为可查询的关系数据库（AST/CFG/数据流/调用图），以基于 Datalog 的面向对象查询语言 QL 编写安全查询，支持过程间污点追踪[26]。

解决问题：以声明式查询表达自定义漏洞模式，支撑 GitHub 官方代码扫描。

实测效果/落地：查询库含数千条社区安全查询，成为安全研究的事实标准之一[26]。

局限性：C/C++ 分析需先构建 CodeQL 数据库，交叉编译工程适配成本高；CLI 许可禁止作为商业竞品分析引擎；深度查询在大仓库耗时显著。

2.2.3 Semgrep（2020）

出处：前身是 Facebook 内部工具 sgrep，由 r2c 公司开源并于 2020 年改名 Semgrep[27]。

技术核心：“规则即代码”——以近似源码形态的模式在 AST 上匹配，附带轻量污点模式，无需完整构建[27]。

解决问题：安全工程师快速编写与维护自定义规则并嵌入 CI 门禁。

实测效果/落地：2025 年因规则许可收紧，社区分叉出 OpenGrep，侧面反映其生态影响力[27]。

局限性：深度跨文件/过程间数据流弱于 CodeQL 与 Coverity（强版本为商业引擎）；模式级匹配对需要路径敏感推理的缺陷无能为力。

2.2.4 Infer（2015）

出处：Meta（Facebook）2015 年 6 月开源，MIT 许可；方法论文发表于 CACM 2015[28]。

技术核心：分离逻辑（Separation Logic）加双向 abduction 的组合式过程间分析：按函数生成摘要并增量组合，无需全程序一次跑完[28]。

解决问题：在万人级工程师的移动开发流程中做到每次提交实时分析。

实测效果/落地：官方口径约 80% 的移动应用发布部署中检出至少一个真实缺陷[28]。

局限性：摘要式组合以覆盖面换可扩展性，跨组件复杂逻辑漏洞覆盖弱；检查器类别相对固定，对 ISR/硬件语义无建模。

2.2.5 SonarQube（2008）

出处：SonarSource 公司（瑞士日内瓦，2008 年创立）[29]。

技术核心：规则引擎加代码度量的持续检查平台，以质量门禁（Quality Gate）嵌入 CI/CD[29]。

解决问题：团队级代码质量管理与技术债治理。

实测效果/落地：开源代码质量平台的事实标准之一，2025 年实证研究显示质量门禁被广泛采用[29]。

局限性：深度污点与安全检测集中于商业版；对 C/C++ 复杂指针场景的分析深度不足，不面向安全关键合规。

2.2.6 Clang Static Analyzer（随 LLVM 演进）

出处：LLVM/Clang 项目内置分析器，Apache-2.0（LLVM 例外条款）[30]。

技术核心：路径敏感的符号执行引擎加可插拔 checker 架构，scan-build 为命令行包装[30]。

解决问题：编译期免费获得空解引用、UAF 等路径敏感检查，被 Chromium 等大型项目采用。

实测效果/落地：随 Xcode 分发，是苹果生态默认静态分析设施[30]。

局限性：跨翻译单元（Cross Translation Unit, CTU）分析仍为实验性且默认关闭，单编译单元视野造成漏报；大项目依赖完整编译数据库且速度慢于 Cppcheck（对应 P1）。

2.2.7 PVS-Studio（2008）

出处：俄罗斯 PVS-Studio 公司，2008 年创立[31]。

技术核心：1,000 余条诊断规则的模式与抽象解释混合分析，以消息分级与多层抑制机制控制误报[31]。

解决问题：商业项目的广谱缺陷检测，长期以免费分析开源项目扩大影响。

实测效果/落地：曾对无人机自驾仪固件 ArduPilot 等航空相关开源项目出具分析报告[31]。

局限性：专有闭源；路径敏感与全程序深度弱于 Coverity；其厂商自述的检出率对比数据缺乏第三方复核。

2.2.8 Cppcheck（2009）

出处：Daniel Marjamäki 发起的开源项目，2009 年 3 月创建仓库，GPL-3.0[32]。

技术核心：自研轻量前端加值流（value flow）跟踪，定位“编译器查不到的缺陷”，以低误报为设计哲学[32]。

解决问题：无需完整构建即可对 C/C++（含嵌入式平台参数 --platform）做快速检查，并提供官方 MISRA 检查插件。

实测效果/落地：GitHub 6,700 余星，嵌入式社区使用最广的开源静态检查器[32]。

局限性：跨翻译单元分析深度受 --max-ctu-depth 限制且主要覆盖浅层检查项，缺乏跨函数深层数据流；无路径敏感符号执行与成熟污点分析；手册明确记载多线程运行且未配置构建目录时全程序检查会静默失效；自研前端对复杂宏与模板的处理与真实编译器存在偏差（本系统预筛选层选型依据与已知短板）。

2.2.9 TscanCode（2015/2016）

出处：腾讯开源，仓库 2015 年 3 月创建，社区通常记 2016 年对外发布[33]。

技术核心：早期基于 Cppcheck 二次开发、后转自研引擎，覆盖 C/C++、C#、Lua，无需编译目标代码[33]。

解决问题：腾讯内部游戏与客户端 C++ 项目的空指针、越界、未初始化检查。

实测效果/落地：厂商口径约 20 万行/分钟、报告准确率约 90%（无第三方复核）[33]。

局限性：未核实到内置 MISRA 支持[待核实]；仓库采用自定义许可声明（非标准 SPDX）；2023 年底后停止更新，腾讯主力转向代码分析平台 TCA。

2.2.10 AFL++（2019）与 OSS-Fuzz（2016）

出处：AFL++ 是 lcamtuf 的 AFL 停更后社区维护的续作（2019 年创建仓库），方法论文发表于 USENIX WOOT 2020[34]；OSS-Fuzz 为 Google 2016 年 12 月上线的持续模糊测试基础设施[35]。

技术核心：覆盖率引导的遗传变异模糊测试（AFL++ 融合 RedQueen、MOPT 等成果）；OSS-Fuzz 以 ClusterFuzz 规模化调度 libFuzzer/AFL++[34][35]。

解决问题：动态侧的真实触发漏洞发现。

实测效果/落地：OSS-Fuzz 官方 README 口径累计发现 13,000 余个安全漏洞、50,000 余个缺陷，覆盖 1,000 余个项目[35]。

局限性：需要源码插桩或二进制重写；对依赖硬件外设输入的嵌入式目标，激励生成难以覆盖 MMIO 状态空间，fuzzing harness 的构建成本高（对应 P7，为第三章动态通道设计的问题起点）。

2.2.11 固件动态分析研究：Firm-AFL（2019）、P2IM（2020）、HALucinator（2020）、μAFL（2022）、Karonte（2020）

Firm-AFL（USENIX Security 2019，中科院郑亚文组）：首个面向 IoT 固件的高吞吐灰盒模糊测试，以“增强进程仿真”融合系统模式仿真（高兼容性）与用户模式仿真

（高吞吐率），约 489 次引用，代码开源[88]。局限性：面向 POSIX 兼容固件，裸机与深度嵌入式目标的外设交互场景覆盖不足。

P2IM（USENIX Security 2020）：自动推断外设接口模型（Peripheral Interface Modeling），使固件无需真实硬件即可在仿真中完成 I/O，并以覆盖引导支持 AFL 模糊测试，代码开源[89]。局限性：模型依靠运行时观测自动推断，依赖固件执行轨迹且冷启动开销大，未被执行到的外设无法建模。

HALucinator（USENIX Security 2020）：以硬件抽象层（Hardware Abstraction Layer, HAL）函数替换实现固件 re-hosting，免除外设仿真[90]。局限性：需人工准备 HAL 模型库，覆盖面取决于模型库积累。

μAFL（ICSE 2022）：面向 MCU 固件外设驱动代码的模糊测试，承袭 P2IM/HALucinator 的外设建模思路并针对 MCU 场景优化[91]。局限性：目标集中于驱动代码，不覆盖 ISR 并发语义与规范符合性维度。

Karonte（IEEE S&P 2020）：面向多二进制固件的协作交互缺陷检测（静态+动态结合）[95]。局限性：聚焦多固件通信安全，不处理单体内部的运行时错误证明。

小结：固件动态分析研究已证明“无硬件执行固件”在技术上可行，但现有工作均以触发为目标、各自独立解决外设建模问题，未与静态通道及运行时监控联动，也未利用厂商 CMSIS-SVD 声明式描述——这正是第三章 3.3.5 与 3.3.6 的增量起点（对应 P7）。

2.2.12 预训练代码模型：CodeBERT（2020）→ GraphCodeBERT（2021）→ UniXcoder（2022）

出处：微软亚洲研究院系列工作：CodeBERT（EMNLP 2020）[36]、GraphCodeBERT（ICLR 2021）[37]、UniXcoder（AAAI 2022）[38]。

技术核心：CodeBERT 以自然语言与编程语言双模态预训练；GraphCodeBERT 引入数据流图预训练任务；UniXcoder 统一 AST 与注释跨模态，兼顾理解与生成[36][37][38]。

解决问题：为漏洞检测等下游任务提供可微调的通用代码表示。

实测效果/落地：发布当时在代码检索与克隆检测等任务上达到最优[36][37][38]。

局限性：上下文窗口小（512 token 级）、函数级粒度，无法覆盖仓库级上下文；在严格去重的 PrimeVul 基准上，此类模型及微调方法的真实检出能力大幅低于早期基准报告值[65]（详见 2.5.3）。

2.2.13 LLM 检测与 LLM 模糊测试方案：GPTScan（2024）、LLM4Vuln（2024）、ChatAFL（2024）、Fuzz4All（2024）与 OSS-Fuzz harness 合成（2024）

GPTScan（ICSE 2024）：将 GPT 与程序分析结合检测智能合约逻辑漏洞：把漏洞拆解为场景加属性，LLM 产生候选后由静态分析确认，精确率超过 90%，静态确认环节削减约三分之二误报[39]。局限性：面向 Solidity 而非 C/C++，依赖外部 API。

LLM4Vuln（2024，arXiv 技术报告）：解耦 LLM 漏洞推理能力与知识获取、上下文检索、提示方案的统一评测框架；在 4 个漏洞赏金项目中发现 14 个零日漏洞[40]。局限性：基准规模较小，无嵌入式知识注入。

ChatAFL（NDSS 2024）：以 LLM 提取协议报文语法并引导有状态协议模糊测试[92]。局限性：面向网络协议，不处理固件 MMIO 输入空间。

Fuzz4All（ICSE 2024）：首个以 LLM 为生成与变异引擎的万能 fuzzer，覆盖多种输入语言与特性[93]。局限性：面向语言与编译器类目标，未针对嵌入式固件。

OSS-Fuzz harness 合成（2024-05，Google 官方上线）：以 LLM 为尚未被 fuzzing 覆盖的开源项目自动生成 harness，已规模化部署[94]。局限性：面向 POSIX 构建的开源项目，依赖可插桩构建，不适用于裸机与 RTOS 固件。

小结：“LLM 参与 fuzzing”在通用生态已被验证并规模化[92][93][94]，本系统的增量不在该机制本身，而在其接地信息（以 SVD 输入域与 ISR 入口表替代纯代码上下文）与执行载体（仿真承载裸机/RTOS 目标），详见 3.3.5。上述方案的另一共同启示是“LLM 生成、程序分析验证”的耦合范式（对应 P5 的解法）。

2.3 B 类：嵌入式/C-C++ 专项工具

2.3.1 Astrée（2009 年商用）

出处：Cousot 夫妇与 Miné 等于 2001—2003 年在巴黎高等师范学院研制，2009 年 2 月由 AbsInt 公司在 ENS/CNRS 许可下商用[45]。

技术核心：抽象解释（Abstract Interpretation）：对变量值域做保守超逼近并迭代收敛，可自动证明程序不存在运行时错误[45]。

解决问题：安全关键同步 C 程序（飞控类）的运行时错误完全性证明。

实测效果/落地：2003 年对 Airbus A340 主飞控软件（约 40 万行 C）实现零误报的运行时错误证明，后应用于 A380 电传飞控[45]。

局限性：面向无递归、无动态分配的同步代码最有效；复杂指针与大循环需要按项目做域（domain）定制，换项目常需专家调参；商业闭源、价格昂贵（对应 P4 的成本侧）。

2.3.2 Polyspace（2007 年并入 MathWorks）

出处：前身 PolySpace Technologies（法国，1999 年）商业化抽象解释技术，MathWorks 于 2007 年 4 月收购[46]。

技术核心：Bug Finder（缺陷与规则检查）加 Code Prover（抽象解释证明所有控制流与数据值下无关键运行时错误）[46]。

解决问题：汽车 ISO 26262、航空 DO-178C 场景的运行时错误证明与合规，提供 DO/IEC 工具鉴定套件[46]。

实测效果/落地：被整车厂与 Tier1 供应商用于 ASIL 项目合规（厂商公开案例）[46]。

局限性：Code Prover 的灰色（未证明）结果需大量人工评审；分析耗时随规模激增，实践中只用于关键模块；证明结论依赖建模假设的正确性（对应 P4）。

2.3.3 Frama-C（2008）

出处：法国原子能与可替代能源委员会-list（CEA List）与 Inria 联合开发，2008 年首次公开发布，核心 LGPL-2.1[53]。

技术核心：可插拔 C 分析平台：EVA 插件做抽象解释值分析，WP 插件以最弱前置条件生成 ACSL 契约的证明义务并交由 SMT 求解器/Coq 消化，E-ACSL 插件将 ACSL 断言编译为运行时检查[53]。

解决问题：从自动值域分析、演绎验证到运行时断言检查的连续谱覆盖，Airbus 与达索曾以其开展 DO-178C 低层需求验证实验[53]。

实测效果/落地：欧洲学术与工业界安全关键 C 验证的主力开源平台[53]。

局限性：WP 演绎验证需要大量人工书写 ACSL 标注；EVA 在无标注大程序上精度下降需调参；无厂商背书的工具鉴定文档（对应 P3、P4）。

2.3.4 CodeSonar（2003）

出处：GrammaTech（康奈尔大学衍生公司）2003 年首发；2023 年其分析业务被收购成立 CodeSecure，2025 年 6 月与 AdaCore 合并[47]。

技术核心：全程序过程间分析（源码与二进制双通道），检测内存破坏、泄漏与并发缺陷[47]。

解决问题：航空起源的大型 C/C++ 深度缺陷检测，NASA 曾将其引入航天网络地面系统分析[47]。

实测效果/落地：安全关键行业长期商用（厂商与 NASA 案例口径）[47]。

局限性：全程序分析的时间与内存开销大，大型库需分片；告警量大导致分诊负担；商业许可费用高。

2.3.5 Klocwork（2001）

出处：2001 年自北电网络（Nortel Networks）分立（渥太华），2014 年入 Rogue Wave，2019 年随其并入 Perforce[48]。

技术核心：数据流/控制流 SAST 加 CWE 安全检查，深度 IDE 集成为长期卖点[48]。

解决问题：嵌入式大型 C/C++ 代码库的持续缺陷检查与合规报告。

实测效果/落地：汽车与医疗行业广泛商用（厂商口径）[48]。

局限性：分析深度介于规则检查器与全程序证明器之间；误报需人工分诊；商业闭源。

2.3.6 Helix QAC（1986/2018）

出处：原英国 Programming Research 公司的 QA·C/QA·C++（1986 年起），2018 年 5 月被 Perforce 收购[49]。

技术核心：以编码标准合规为核心的静态分析，按 MISRA Compliance 框架输出合规记录，是 MISRA 合规的事实标准工具之一[49]。

解决问题：汽车/航空行业的编码规范强制合规。

实测效果/落地：长期被整车厂用于 ISO 26262 项目（厂商口径）[49]。

局限性：不做全程序路径证明，运行时错误覆盖能力弱于抽象解释类工具；对逻辑缺陷检出有限；商业闭源。

2.3.7 LDRA（1975）

出处：Liverpool Data Research Associates，1975 年成立，业界最早的静态分析/验证工具厂商之一[50]。

技术核心：Testbed 静态与动态分析、编码标准与覆盖率一体，TBrun 提供单元测试与自动打桩，支持 DO-178 的数据/控制耦合分析[50]。

解决问题：航空与国防供应链的全套验证证据。

实测效果/落地：其套件可鉴定为 DO-178 软件验证工具，长期服务 NATO 供应链（厂商口径）[50]。

局限性：公开的第三方对比数据少；工作流偏传统，现代 CI 体验一般；商业授权昂贵。

2.3.8 Parasoft C/C++test（1999）

出处：Parasoft 公司 1987 年创立，C/C++test 产品 1999 年推出[51]。

技术核心：静态分析（MISRA/AUTOSAR/CERT/CWE 规则）加自动单元测试生成与运行时检测（Insure++）的整合套件[51]。

解决问题：嵌入式 C/C++ 安全关键开发的规则合规与测试补充。

实测效果/落地：自称服务财富 500 强三十余年（厂商口径）[51]。

局限性：自动生成测试的断言深度有限；误报需调优；套件体量大、学习成本高。

2.3.9 VectorCAST（1990）

出处：Vector Software 1990 年创立，2017 年 1 月并入德国 Vector 集团[52]。

技术核心：基于 V 模型的单元/集成测试自动化：自动生成驱动与桩、回归管理、MC/DC 覆盖率，提供 DO-178B/C/DO-330 鉴定包[52]。

解决问题：动态侧的认证级测试执行与覆盖率证据。

实测效果/落地：航空业广泛商用（厂商口径）[52]。

局限性：解决测试执行自动化而非静态证明；许可与维护成本高；测试生成质量依赖被测代码可测性。

2.3.10 ESBMC（2011）

出处：Lucas Cordeiro 与 Bernd Fischer 等，CAV 2011 首发；初代复用 CBMC 前端，v5.0（2018）起改用 Clang 前端[54]。

技术核心：基于 SMT 的有界模型检查（Bounded Model Checking, BMC），支持 k-归纳、并发（数据竞争与死锁）、浮点精确模式与反例生成[54]。

解决问题：开源侧兼具并发与浮点能力的 C/C++ 模型检查。

实测效果/落地：SV-COMP 多赛道常年参赛，2025 年竞赛报告显示其在并发安全赛道的稳定增益[54][60]。

局限性：BMC 完备性受展开界限制（未证到不等于无缺陷）；深循环与大状态空间下 SMT 求解开销大；并发建模对锁模式依赖较强。

2.4 C 类：形式化方法与符号执行

2.4.1 CBMC（2004）

出处：Clarke、Kroening 与 Lerda，TACAS 2004；现由 AWS 自动推理团队主导维护[55]。

技术核心：位精确的 C 程序有界模型检查：程序按界展开后编码为 SAT/SMT 公式，检查断言、溢出、指针安全与 UB，v6 起默认开启 UB 检查[55]。

解决问题：对关键性质给出界内的完备判定与反例。

实测效果/落地：AWS 在关键基础设施项目中用其做穷举性质验证（Proving Grounds 项目口径）[55]。

局限性：证明范围受展开界限制；状态空间随展开深度指数增长；对嵌入式工程需要可运行的展开驱动（harness）与外设建模（对应 P2、P4、P7）。

2.4.2 KLEE（2008）

出处：Cadar、Dunbar 与 Engler（斯坦福），OSDI 2008[56]。

技术核心：基于 LLVM 的符号执行（Symbolic Execution）引擎：以路径约束求解自动生成高覆盖测试，含约束缓存与查询优化[56]。

解决问题：无人干预地为系统程序生成高覆盖测试并挖掘深层缺陷。

实测效果/落地：在 GNU Coreutils 上的行覆盖平均超过 90%（远高于手写测试套件约 68%），发现 56 个真实缺陷[56]。

局限性：路径爆炸限制深循环覆盖；系统调用与环境建模不完整；目标需重编译至 LLVM IR，交叉编译工程适配成本高；对浮点与并发支持有限。

2.4.3 CPAchecker（2011）

出处：Dirk Beyer 与 Keremoglu，CAV 2011（注意：发表于 CAV 而非 TACAS）[57]。

技术核心：可配置程序分析（Configurable Program Analysis, CPA）框架：谓词抽象、显式值分析、BAM 模块化分析等算子可组合切换，支持见证（witness）生成与校验[57]。

解决问题：软件验证的通用可达性与内存安全检查。

实测效果/落地：SV-COMP 2012 首届总冠军并多年多赛道夺冠，2025 届仍获可达安全与内存安全赛道冠军[57][60]。

局限性：最优精度依赖配置选择；对并发 C 与复杂位运算/浮点支持弱于专用工具；反例定位质量一般。

2.4.4 IKOS（2015）

出处：NASA 艾姆斯研究中心，2015 年发布[58]。

技术核心：基于抽象解释的通用 C/C++ 分析框架（LLVM 前端），检查缓冲区溢出、除零、未初始化、空解引用等运行时错误[58]。

解决问题：NASA 飞行软件等安全关键代码的运行时错误分析。

实测效果/落地：NASA 内部使用并开源[58]。

局限性：默认抽象域对真实工业代码精度偏低（大量“可能为真”告警）；不支持并发分析；社区规模小、更新慢。

2.4.5 Goblint（2007）

出处：Vojdani 与 Vene，SPLST 2007（起点为 2007 年而非 2017 年），现由慕尼黑工业大学主导[59]。

技术核心：基于抽象解释与线程模块化的 C 静态分析器，专长多线程程序的数据竞争健全检查[59]。

解决问题：并发 C 程序的健全数据竞争检测。

实测效果/落地：SV-COMP 数据竞争赛道 2021、2023、2024 届得分最高[59][60]。

局限性：检查类别相对专窄；健全但精度依赖域设计；其线程模型基于 pthread 共享内存假设，对 ISR/RTOS 任务语义仍需上层适配（对应 P2）。

2.4.6 SV-COMP（2012）

出处：TACAS 工具竞赛（Beyer 组织），2012 年首届，2026 年为第 15 届[60]。

技术核心：统一的验证任务集与见证校验闭环评分，覆盖可达安全、内存安全、溢出、并发、终止等类别[60]。

解决问题：软件验证工具的横向可比评测。

实测效果/落地：2025 届 34,700 个验证任务；2026 届 74 个验证工具加 20 个校验器参赛[60][69]。

局限性：任务多为小规模单性质程序，与工业规模代码差距大；存在面向基准调参的争议（评测外推需谨慎，对应 P6）。

2.5 D 类：学术里程碑

2.5.1 规则挖掘与属性验证时代（2001—2005）：Metal、ESP、MOPS、Saturn

Metal（PLDI 2001，斯坦福 Engler 组）：meta-level compilation——程序员以 xcc 语言把系统规则写成编译器扩展，在编译 Linux 内核与 FLASH 协议栈时同步检查，发现数百量级真实缺陷[41]；其“缺陷即异常行为”的统计挖掘思想（SOSP 2001）影响深远。局限性：规则需逐条人工书写，属性覆盖依赖专家投入。

ESP（PLDI 2002，微软研究院 Das 等）：以路径模拟（path simulation）折叠控制流图中可互模拟的路径，在多项式时间内实现路径敏感验证[42]。局限性：面向特定验证属性，需人工指定目标。

MOPS（CCS 2002，伯克利 Chen 与 Wagner）：把程序建模为下推自动机、安全属性写成有限状态机，模型检查所有过程间路径（如特权操作序列）[43]。局限性：仅能表达有限状态属性，逐属性建模成本高。

Saturn（POPL 2005，斯坦福 Xie 与 Aiken）：以 SAT 求解器驱动的路径敏感、位精确分析，自底向上为函数生成布尔摘要再组合，配套 CAL 分析语言；应用于 Linux 内核检出未知缺陷[44]。局限性：SAT 求解开销大、规则种类少，项目早已停止维护。

小结：这一代工作确立了“属性驱动的系统代码检查”范式，其规则需人工书写的瓶颈后来由统计学习与大模型接续——这正是本系统以规则库为骨架、以 LLM 降低规则与研判成本的思想源头。

2.5.2 深度学习时代（2018—2021）：VulDeePecker、Devign、SySeVR、D2A

VulDeePecker（NDSS 2018，华中科技大学李珍组）：围绕安全敏感 API 调用做程序切片（code gadget），以双向 LSTM 分类；在 Xen、Seamonkey、Libav 中发现 4 个未被 NVD 收录的漏洞[61]。局限性：仅覆盖依赖 API 特征的两类漏洞；切片截断损失长函数上下文。

Devign（NeurIPS 2019，南洋理工大学杨易组/IBM）：把函数表示为 AST+CFG+DFG 复合图，以门控图神经网络做图级分类；发布 FFmpeg+QEMU 人工标注数据集（27,996 个函数）[62]。局限性：后续独立重评测显示其在其他真实数据集上 F1 大幅下滑，暴露基准过拟合与标注噪声问题[76]。

SySeVR（IEEE TDSC 2022，李珍组）：将切片从 API 调用泛化为四类语法构件（SyVC）到语义切片（SeVC）的系统框架[63]。局限性：切片以语法元素为中心，跨文件上下文不足；同样受数据集泄漏影响。

D2A（ICSE-SEIP 2021，IBM）：用商用静态分析器对开源项目漏洞修复前后版本做差分标注，自动构建 130 万余个函数级标注样本[64]。局限性：标签继承该分析器自身的漏报/误报偏差，存在系统性噪声[64]。

2.5.3 大模型与严格基准时代（2024—2025）：PrimeVul、VulBench

PrimeVul（ICSE 2025，威斯康星大学麦迪逊分校 Ding 等）：以严格去重、按时间划分与配对评估构建的 C/C++ 函数级基准（约 23 万函数、6,968 个漏洞函数、140 余种 CWE）；实证揭示此前基准严重高估：在严格设置下，SOTA 7B 模型的 F1 从 Big-Vul 上的 68.26% 跌至 3.09%，GPT 级模型接近随机猜测[65]。

VulBench（NAACL 2024，华中科技大学 Gao 等）：聚合 CTF 挑战与真实 CVE 数据，评测 16 个 LLM 与传统深度学习/图方法，发现部分 LLM 优于传统基线，且补充函数间上下文可提升性能[66]。

对本案的直接启示：其一，任何 LLM 检测方案必须在严格去重与时间划分的基准上报告结果，否则结论不可信[65]；其二，函数级孤立判断远不充分，仓库级上下文与过程间信息是提升 LLM 检出能力的关键变量[66]；其三，这正是本系统把 LLM 定位为“协同中枢而非最终裁决”、并以工具验证约束其输出的实证依据。

2.6 为什么通用工具在嵌入式场景水土不服

综合 2.2 至 2.5 的逐项局限，通用工具在嵌入式场景的失效可归纳为八个相互交织的原因，每一条都对应可核查的技术事实，而非泛泛的经验之谈。

（1）编译器方言与内联汇编。嵌入式代码大量使用编译器扩展（__attribute__((interrupt))、#pragma location、内联汇编）。自研前端类工具（如 Cppcheck）对非标准语法的解析与真实编译器存在偏差，轻则误报、重则整文件解析失败跳过。

（2）内存映射寄存器与 volatile。对外设地址的读写与 volatile 语义是 C 标准中最难精确建模的部分，通用工具将其视为普通指针访问，导致两类后果：围绕寄存器访问的正确性检查（只读寄存器被写、位域越界）缺失；volatile 变量上的“冗余读写”类告警成为高频误报。

（3）ISR 与主循环的并发。通用数据竞争检测建立在 pthread 线程模型上（如 Goblint[59]），而嵌入式的并发单位是 ISR 与 RTOS 任务，竞争关系由中断优先级与 RTOS API 语义决定，通用模型对此系统性漏报。

（4）无标准库与裸机启动代码。裸机工程缺少完整标准库与 main 之上的运行时，通用工具以“main 为入口、库函数语义已知”的假设失效，摘要分析从入口即不完整。

（5）交叉编译构建环境缺失。CodeQL 需构建数据库、Coverity 需构建捕获、Clang 系需 compile_commands.json，而嵌入式工程的构建脚本（make/CMake/IAR 工程）往往无法在分析环境复现，导致工具导入即失败（对应 P1，为工业落地的第一道门槛）。

（6）第三方 SDK 闭源。芯片厂商 SDK 常以二进制库加头文件发布，过程间分析在库边界中断，调用链不完整，漏报与误报同时增加。

（7）RTOS 语义不可见。任务创建、队列、信号量、延时等 RTOS API 的调度与阻塞语义对通用分析器是普通函数调用，死锁、优先级反转、阻塞调用误用等缺陷不可见（对应 P2）。

（8）性能与增量分析需求。嵌入式工程频繁全量重建，全程序深度分析（CodeSonar、Astrée 类）单次数小时，无法嵌入每日集成；缺乏基于调用图影响集的增量能力使深度分析在嵌入式 CI 中不可用。

归纳：八个原因中，（1）—（4）是“语义建模缺失”，（5）是“工程环境缺失”，（6）—（7）是“上下文缺失”，（8）是“工程化缺失”。第三章的 P1—P7 即按此归纳并逐条对位；动态侧的对应障碍（插桩检测环境依赖、harness 成本、MMIO 激励空间）已由 1.2 节与 2.2.10/2.2.11 给出，归入 P7。

2.7 大模型+嵌入式检测的参考项目

除学术工具外，社区中已出现若干“LLM+嵌入式检测”方向的工程实践。本节逐项核实其现状（核实时间为 2026 年 10 月），作为本系统的参照系；无法核实到公开来源的项目显式标注。

2.7.1 LCMHalMCP[未检索到]

状态说明：据项目前期材料记录，该项目将 CodeQL 静态分析、MCP 工具接口、LangGraph 多智能体编排、源码替换、交叉编译与 Unicorn/fuzzemu 仿真串成闭环，支持 STM32、NXP i.MX RT、Zephyr 平台与裸机、FreeRTOS、RT-Thread 运行时。2026 年 10

月以仓库名与作者名公开检索未能核实到该仓库（可能私有、改名或已删除），其描述按提供方材料收录[待核实]，立项后应向材料提供方索取地址核实。

参考价值：即便暂不可核实，其“检测（CodeQL）-决策（多智能体）-验证（仿真）”闭环的架构描述与本系统三层设计同构，仍是工具链协同的重要参照。

2.7.2 embedded-ai-workspace（已核实）

出处：github.com/atillab1/embedded-ai-workspace，TypeScript 实现，在 STM32F4-DISCOVERY 硬件上完成端到端验证[97]。

技术核心：三阶段 Claude 工具链：MISRA-C:2012（含 AMD4，覆盖 ISR 与 DMA 陷阱）审查 Skill（纯推理层，不运行代码）；Datasheet MCP（对 RM0090 参考手册做向量检索）；Serial MCP（USB UART 硬件在环交互）[97]。

参考价值：其关键设计洞察——Skill 改变模型的行为方式、MCP 扩展模型的能力边界——为本系统划定 LLM 参与形式（3.3.7/3.3.8：行为类能力沉淀为提示词层，能力类扩展实现为工具层）提供了直接依据。

局限性：个人项目、社区规模小（misra-c 话题下个位数 star 量级）；面向单芯片（STM32F4）验证；无检测结果可证明性分级与规范合规记录输出。

2.7.3 中科院软件所嵌入式漏洞挖掘技术（非公开）

状态说明：据行业交流材料，该技术处于中试阶段：多层次上下文提示结合 AST 语义抽取实现片段、模块、项目三级缺陷检查，修复阶段采用分析-生成-验证-优化四步流程。无公开论文或代码，按提供方材料收录[待核实]。

参考价值：三级检查粒度划分（片段到项目）与本系统静态预筛选到 LLM 研判的分层思路互相印证。

2.7.4 通用对照基线：open-code-review（2026）

出处/数据：alibaba/open-code-review，2026 年 9 月实测约 42,795 星，Apache-2.0，确定性流水线加 LLM 智能体混合架构[98]。

参考价值：作为通用 AI 代码审查的效果对照基线：本系统的评测（3.5 节）需证明其在嵌入式场景上优于通用方案，差异化结论才有说服力。

2.8 横向对比总表与解读

下表将 2.2 至 2.5 节调研的 31 组工具与项目按统一维度汇总，是第三章问题定义（P1—P7）与方案对标的唯一落脚点。出处列对应参考文献编号；嵌入式适配一列综合考量 MISRA/中断/RTOS/交叉编译四项；可证明性指该工具能否给出健全（sound）证明或界内完备结论。

表 2-1 嵌入式代码检测相关工具与项目横向对比总表（星标类动态数据截至 2026 年 10 月）

工具/项目	年份	出处	核心技术	语言覆盖	缺陷类型	需源码	嵌入式适配	可证明性	误报水平	属性	主要局限
Coverity	2002	[25]	全程序数据流/控制流分析	C/C++/Java 等	内存/泄漏/逻辑	是	需构建捕获，交叉工程常失败	检查器级	中低	商业	构建捕获依赖；定制灵活性低
CodeQL	2019 开放	[26]	代码数据库+QL 查询（Datalog 系）	多语言	安全漏洞/污点	是	需构建数据库，适配成本高	查询可表达	视查询	查询库开源/引擎专有	大库耗时；商用许可限制
Semgrep	2020	[27]	AST 模式匹配+轻量污点	多语言	规则/污点	否	可自定义 MISRA 子集规则	否	低	LGPL-2.1	深度过程间分析弱
Infer	2015	[28]	分离逻辑+abduction 函数摘要	Java/C/C++/ObjC	空指针/泄漏/并发	是	无 ISR/硬件建模	摘要级	低	MIT	跨组件逻辑覆盖弱
SonarQube	2008	[29]	规则+度量+质量门禁	多语言	坏味道/缺陷	否	弱	否	中	LGPL-3.0+商业	C/C++ 深度不足
Clang SA	随 LLVM	[30]	路径敏感符号执行+checker	C/C++/ObjC	空解引用/UAF 等	是	CTU 实验性、默认关闭	路径反例	中	Apache-2.0	跨单元漏报；大库慢
PVS-Studio	2008	[31]	1,000+ 诊断规则+误报抑制	C/C++/C#/Java	广谱缺陷	否	有 ArduPilot 等案例	否	中低	商业	闭源；深度弱于 Coverity

工具/项目	年份	出处	核心技术	语言覆盖	缺陷类型	需源码	嵌入式适配	可证明性	误报水平	属性	主要局限
Cppcheck	2009	[32]	自研前端+值流跟踪	C/C++	UB 类缺陷	否	--platform 参数；MISRA 插件	否	低（保守）	GPL-3.0	CTU 深度受限；无路径敏感
TscanCode	2015/16	[33]	Cppcheck 衍生+自研引擎	C++/C#/Lua	空指针/越界	否	未核实到 MISRA 支持	否	中	自定义声明	已停更（2023 年底）
AFL++ / OSS-Fuzz	2019/2016	[34] [35]	覆盖率引导模糊测试	C/C++ 等	触发型漏洞	是（插桩）	MMIO 激励难构造	—	低（真触发）	AGPL/混合	harness 成本高；外设输入难覆盖
CodeBERT→UniXcoder	2020-22	[36-38]	预训练代码表示	多语言	下游微调任务	是	无	否	高（严格基准下）	MIT	窗口小；PrimeVul 下大幅下滑
GPTScan / LLM4Vuln	2024	[39] [40]	LLM+程序分析 / 受控评测	Solidity/多语言	逻辑漏洞	是	非嵌入式	否	GPTScan P>90%（合约）	开源	领域特定；外部 API 依赖
Firm-AFL/ P2IM/ HALucinator/ μAFL	2019-22	[88-91]	固件仿真+外设建模+模糊测试	C（固件）	触发型漏洞	是（固件镜像）	面向固件 re-hosting，各自独立建模	—	低（真触发）	学术开源	未联动静态通道与运行时监控
ChatAFL/ Fuzz4All/OSS-Fuzz harness 合成	2024	[92-94]	LLM 引导 fuzzing 与 harness 自动生成	协议/多语言	触发型漏洞	是	面向 POSIX/协议目标，非裸机	—	视 oracle	开源	无硬件语义接地
Astrée	2009 商用	[45]	抽象解释（域工程）	C	运行时错误	是	A340/A380 飞控验证	健全证明	零误报（证明类）	商业	需逐项目域调参；闭源昂贵
Polyspace	2007 并入	[46]	抽象解释（Bug Finder/Code Prover）	C/C++/Ada	运行时错误	是	DO/IEC 工具鉴定套件	健全证明	灰色结果需人工	商业	规模受限；依赖建模假设

工具/项目	年份	出处	核心技术	语言覆盖	缺陷类型	需源码	嵌入式适配	可证明性	误报水平	属性	主要局限
Frama-C	2008	[53]	EVA 抽象解释+WP 演绎验证+E-ACSL 运行时断言	C	RTE+契约证明+运行时检查	是	Airbus/Dassault 实验	健全/WP 证明	可调	LGPL-2.1	ACSL 人工标注成本高
CodeSonar	2003	[47]	全程序过程间（源码+二进制）	C/C++/Java	内存/并发	是	航空起源；NASA 案例	部分	中高（告警多）	商业	资源开销大；分诊负担
Klocwork	2001	[48]	SAST 数据流+IDE 集成	C/C++ 等	CWE/缺陷	是	汽车/医疗合规	否	中	商业	深度居中；闭源
Helix QAC	1986	[49]	编码标准合规引擎	C/C++	MISRA/CERT 合规	是	MISRA 合规事实标准	否	低	商业	逻辑缺陷检出有限
LDRA	1975	[50]	静态+动态+覆盖率一体	C/C++/Ada	标准+覆盖率	是	DO-178 工具鉴定	部分	中	商业	CI 体验一般；第三方数据少
Parasoft C/C++test	1999	[51]	静态分析+自动单测	C/C++	规则+测试	是	MISRA/AUTOSAR	否	中	商业	断言深度有限
VectorCAST	1990	[52]	单元/集成测试自动化	C/C++/Ada	覆盖率（MC/DC）	是	DO-178 鉴定包	—（动态）	—	商业	不替代静态证明
CBMC	2004	[55]	位精确有界模型检查	C/C++	断言/溢出/UB	是	AWS 应用；需 harness	界内完备	低（带反例）	BSD 系	展开界限制完备性
ESBMC	2011	[54]	SMT BMC+k-归纳+并发	C/C++/SystemC	并发/浮点/内存	是	SV-COMP 多赛道	k-归纳证明	低	BSD 系	深循环求解开销大
KLEE	2008	[56]	LLVM 符号执行	C/C++	深路径+测试生成	是	需重编译至 LLVM IR	反例	低	BSD 系	路径爆炸；环境建模不完整
CPAchecker	2011	[57]	可配置 CPA（谓词/显式值）	C	可达性/内存	是	SV-COMP 多届冠军	健全	可调	Apache-2.0	配置复杂；并发支持弱
Goblint	2007	[59]	抽象解释+线程模块化	C	数据竞争	是	基于 pthread 假设，ISR 需适配	健全	中	MIT	检查类别窄
IKOS	2015	[58]	抽象解释框架（LLVM 前端）	C/C++	运行时错误	是	NASA 飞行软件	健全（默认域）	中高	NASA OSA	默认精度低；不支持并发

工具/项目	年份	出处	核心技术	语言覆盖	缺陷类型	需源码	嵌入式适配	可证明性	误报水平	属性	主要局限
VulDeePecker/ Devign/ SySeVR/D2A	2018-21	[61-64]	程序切片/图神经网络/差分标注	C/C++	函数级漏洞	是	无嵌入式感知	否	基准泄漏致虚高	学术开源	标注噪声；跨数据集泛化差
PrimeVul / VulBench	2024-25	[65] [66]	严格去重基准	C/C++	评测	是	无硬件上下文	—	—	开源	揭示既有模型接近随机猜测

总表解读：横向对比揭示三个空白与一个趋势。空白一：嵌入式适配维度上，商业工具（Astrée、Polyspace、QAC、LDRA）以合规见长但闭源昂贵且逐项目定制，开源工具（Cppcheck、CSA、CBMC/ESBMC）可获取但普遍缺失 ISR 并发、寄存器语义与 RTOS 建模——两列能力在开源侧几乎空白。空白二：可证明性维度上，能给出健全证明的工具（Astrée/Polyspace/Frama-C/CPAchecker）与能高效接入工程实践的工具（Cppcheck/Semgrep）分居谱系两端，缺少“按结论强度分级输出”的中间层。空白三：数据维度上，所有开源可获取的数据集均不带嵌入式硬件上下文标注（最后两行），自建数据集具备明确增量。动态侧：模糊测试基础设施（第 10 行）与固件动态分析研究（第 13 行）分别在通用与固件生态战果辉煌，但 harness 成本与 MMIO 激励两道墙使其在嵌入式工程落地稀少，且与静态通道、运行时监控互不联动。趋势上，LLM 方案（第 12、14 行）与严格基准（第 31 行）的对照表明：LLM 的语义能力真实存在但必须以程序分析验证为约束——这正是第三章“双通道为骨架、多智能体为中枢、工具验证为约束”混合式路线的立论。

第三章　问题定义与技术方案

3.1 急需解决的问题

基于第二章逐项核实的局限性证据，本节凝练七个亟需解决的问题（P1—P7）。每条问题均明确追溯到第二章的具体工具缺陷或第一章的事实约束，形成“别人不行、所以我要做”的论证闭环；3.3 节的每个模块将对应回这些问题编号。

P1 构建环境缺失导致静态分析跑不起来。嵌入式工程的编译由交叉工具链与分散配置（make/CMake/IAR/Keil 工程、板级宏定义、SDK 头文件）构成，分析环境往往无法复现。直接后果是主流引擎的接入门槛：Coverity 依赖完整构建捕获[25]、CodeQL 需要先构建数据库[26]、Clang Static Analyzer 依赖编译数据库且跨单元分析默认关闭[30]、Cppcheck 自研前端对复杂宏的解析与真实编译器存在偏差[32]。若不解决，后续一切分析无从谈起。

P2 中断/并发/硬件语义建模缺失，是通用引擎的最大盲区。嵌入式并发单位是 ISR 与 RTOS 任务，竞争关系由中断优先级与 RTOS API 语义决定；而通用竞争检测建立在 pthread 线程模型之上（如 Goblint[59]），Infer 等 SAST 引擎对 ISR 无建模[28]，IKOS 明确不支持并发[58]。MMIO 寄存器访问与 volatile 语义同样缺乏显式模型，导致寄存器相关缺陷不可检、volatile 类告警成为高频误报（2.6 节原因（2）（3）（7））。

P3 规范符合性（MISRA/GJB）与漏洞检测两套工具割裂。编码标准合规由 QAC、LDRA 等合规引擎承担[49][50]，漏洞检测由 Coverity、CodeQL 等承担[25][26]，两者在规则集、输出格式、证据链与工作流上互不相通；开源侧 TscanCode 未核实到 MISRA 支持[33]，Frama-C 无厂商背书的鉴定文档[53]。企业须同时采购维护两套工具并人工汇编合规报告，成本高且口径不一致。

P4 漏报与误报不可调和，且结果缺乏可证明性分级。健全的抽象解释需要逐项目域工程且灰色结果需人工评审（Astrée[45]、Polyspace[46]）；BMC 完备性受展开界限制（CBMC[55]）；保守的规则类工具低误报但系统性漏报深层缺陷（Cppcheck[32]）。现

有工具输出单一告警流，不区分“数学证明的结论”与“疑似告警”，安全关键场景无法按合规要求使用。

P5 LLM 方案幻觉严重、可解释性差、仓库级上下文受限。严格基准表明 LLM 直接判漏洞接近随机：SOTA 7B 模型 F1 在 PrimeVul 上仅 3.09%[65]；预训练代码模型上下文窗口小、函数级粒度[36][37][38]；现有 LLM 检测方案或局限于非 C/C++ 领域（GPTScan 面向智能合约[39]），或依赖外部 API 且无嵌入式知识注入（LLM4Vuln[40]）。结论无证据链、无工具验证，无法进入安全关键流程。

P6 嵌入式专属数据集与评测基准缺失。Juliet 为人工合成用例，good/bad 配对使模型学到结构捷径[67]；Big-Vul、CVEfixes 等真实数据集来自通用软件，无硬件上下文标注，且存在重复与泄漏导致的高估[65][70][71]；SV-COMP 任务为小规模单性质程序[60]。嵌入式特有缺陷类型（ISR 竞争、寄存器误访问）在任何现有公开基准中均无系统覆盖，导致本领域方法评测失真。

P7 动态检测与运行时监控在资源受限平台难以落地。动态检测的真实触发能力是静态分析无法替代的（OSS-Fuzz 累计发现 13,000 余个安全漏洞[35]），但其在嵌入式场景存在系统性障碍：其一，ASan/TSan 等插桩检测依赖操作系统环境，裸机平台仅 UBSan standalone 可用（1.2 节）；其二，覆盖率引导模糊测试需要 harness 与插桩支持，且对依赖硬件外设输入的目标，激励生成难以覆盖 MMIO 状态空间（2.2.10[34][35]）；其三，千字节级内存目标无法承受全量插桩与 trace 记录的开销。固件动态分析研究（Firm-AFL、P2IM、HALucinator、μAFL[88-91]）已解决“无硬件执行固件”的问题，但均以独立触发为目标、各自解决外设建模，未与静态通道及运行时监控联动。后果：动态检测在嵌入式工程实践普遍缺位，时序、并发时序与真实输入触发类缺陷只能靠现场暴露——而这类缺陷恰是静态分析最不可判定的盲区，两个盲区叠加构成现有方案最大的能力空洞。

3.2 总体技术路线

本系统采用**静态与动态双通道为骨架、LLM 多智能体协同为中枢、工具验证与证据链为约束**的混合式（hybrid）路线。相较仅以静态分析为骨架、LLM 为增强的保守设计，本版做出两点升级并给出证据依据。

升级一：动态分析升格为与静态并列的第一类检测通道，并新增运行时监控能力。理由有三：其一，动态发现是真触发、低误报的，OSS-Fuzz 的 13,000 余个漏洞即为其产能证明[35]，恰好补偿静态通道的误报短板；其二，静态最不可判定的并发时序与硬件交互类缺陷，正是运行时监控可以直接裁决的对象（P2 与 P7 互为出口）；其三，仿真技术使裸机动态检测绕开硬件依赖——固件动态分析研究已证明无硬件执行固件可行（Firm-AFL/P2IM/HALucinator/μAFL[88-91]），Renode 提供外设级仿真[83]、Unicorn 提供轻量 CPU 级仿真[84]、Frama-C 的 E-ACSL 支撑断言的运行时检查[53]，P7 的三道墙（环境依赖、harness 成本、MMIO 激励）逐一有解。

升级二：LLM 从增强层升格为多智能体协同中枢。理由有二：其一，检测任务天然异构——静态告警流、动态触发证据、运行时事件三类产物需要语义级关联与冲突消解，其编排复杂性已超出固定流水线的表达能力；其二，“LLM 驱动、工具验证”的闭环已被实证有效：GPTScan 以静态分析确认 LLM 候选取得超过 90% 精确率[39]，固件智能体工作以 CWE 分类驱动迭代修补并经仿真验证取得 92.4% 修复率[77]，FirmAgent 以模糊测试证据引导 LLM 智能体发现 IoT 固件漏洞[85]。**同时必须划定采纳边界：LLM 的“核心”地位体现在编排与语义推理，不在最终裁决——严格基准下 LLM 直接判定漏洞接近随机（SOTA 7B 模型 F1 仅 3.09%[65]），因此任何进入报告的结论必须携带工具可复核的证据链，LLM 不作最终裁决。**

系统按五层组织，总体架构见图 3-1。L1 编译环境复现层解决 P1：从任意嵌入式工程产出编译数据库与工具链容器。L2 统一 IR/程序表示层将 Clang AST/IR、调用图与 SARIF 告警统一为单一数据模型（P3 的表示统一），并在其上注入硬件语义标注（P2），同时向动态通道输出 harness 所需元数据（函数签名、寄存器模型、中断入口表）。L3 双通道多引擎分析层：静态通道运行规则引擎、抽象解释、符号执行/BMC 与数据流/污点分

析（P2、P3、P4）；动态通道运行覆盖率引导模糊测试、插桩检测与仿真执行，并承载运行时监测点（P7）。L4 多智能体协同层由六类 LLM 智能体（规划、静态研判、动态测试运行时监控、修复、报告）以状态机方式协同（P5、P7），共享工具验证闭环。L5 结果聚合与可证明性输出层聚合去重、映射 CWE/MISRA、整合动态 trace 证据、生成合规报告（P3、P4）。自建数据集（P6）作为评测支撑贯穿各层。

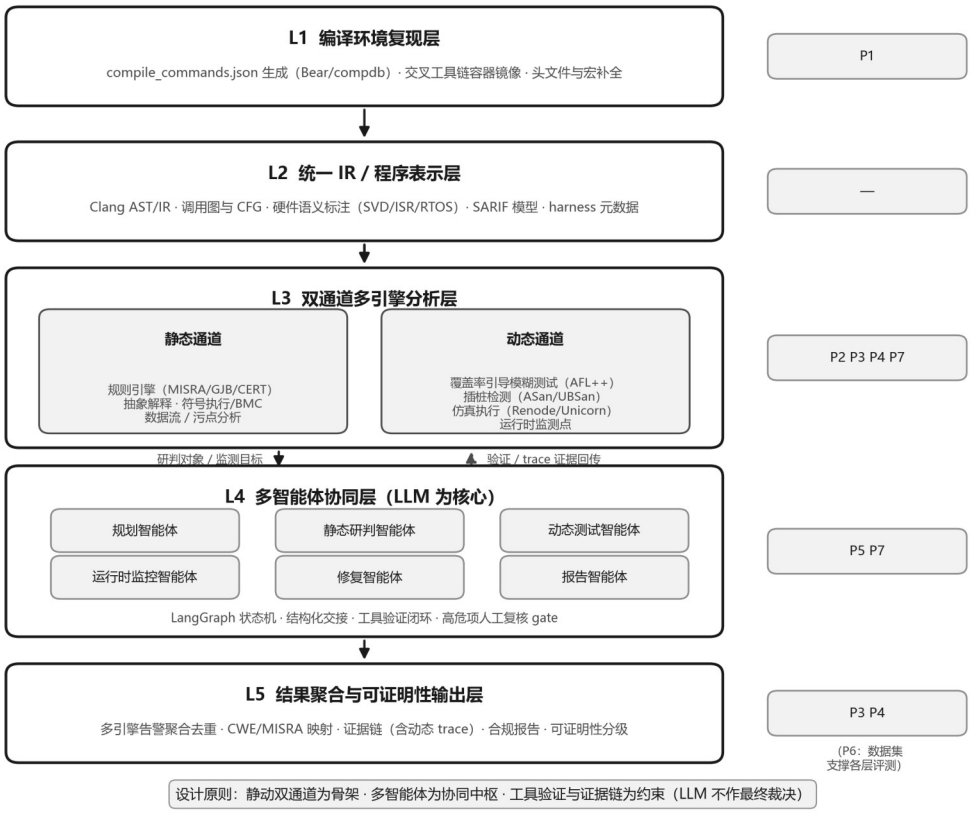


图 3-1 系统总体技术架构（五层；L3 静动双通道，L4 六类智能体协同；右侧标注对应问题编号）

3.3 关键模块的具体实现

本节按 P1—P7 逐条给出模块设计。每个模块写清输入输出、技术选型（区分复用与自研）、关键流程伪代码及其解决问题的机理。表 3-1 给出问题与模块的总映射。

表 3-1 问题（P1—P7）与模块及可复用技术的对应关系

问题	对应模块（3.3 节）	可复用的现有技术
P1 构建环境缺失	3.3.1 编译命令与构建环境自动复现	Bear/compdb[73]、CMake 导出、Clang 交叉驱动
P2 并发/硬件语义缺失	3.3.2 硬件语义与并发建模	CMSIS-SVD[74] + cmsis-svd 解析库[96]、Goblint 线程模块[59]、ESBMC 并发编码[54]
P3 规范与漏洞检测割裂	3.3.3 规则引擎 + 3.3.10 聚合输出	cppcheck MISRA 插件[32]、clang-tidy/Semgrep 框架、SARIF
P4 误报漏报与可证明性	3.3.4 静态深度分析适配	Infer 摘要式分析[28]、ESBMC k-归纳[54]、CPAchecker 域组合[57]
P5 LLM 幻觉与上下文	3.3.7 多智能体协同 + 3.3.8 研判修复机制	GPTScan 生成-验证范式[39]、RAG、LangGraph[86]、vLLM 本地部署
P6 数据集缺失	3.4.3 自建嵌入式数据集	CVEfixes 采集框架[70]、PrimeVul 清洗与划分方法[65]
P7 动态/监控落地难	3.3.5 动态分析引擎 + 3.3.6 运行时监控	AFL++[34]、Renode[83]、Unicorn[84]、E-ACSL[53]、P2IM/HALucinator 外设建模思路[89][90]

3.3.1 编译命令与构建环境自动复现（对应 P1）

输入/输出：输入为目标工程仓库与目标平台标识（如 STM32F4、arm-none-eabi）；输出为与真实构建语义一致的 compile_commands.json、交叉工具链容器镜像与宏/头文件补全清单。

技术选型：对 CMake/Make/qmake 工程，优先使用其原生导出或 Bear（Build EAR）构建拦截生成编译数据库[73]；对 IAR/Keil 等闭源 IDE 工程（.ewp/.uvprojx 为 XML），自研工程文件解析器，将包含路径、宏定义、目标核与字节序翻译为等价 Clang 交叉驱动命令行；工具链镜像以 Docker 固化（gcc-arm-none-eabi、IAR 编译器按许可挂载），保证可复现；宏与头文件补全通过依赖扫描（预处理探测缺失头文件）加 SDK 包索引实现。

```
procedure BuildReconstruct(repo, target):
  if has(repo, 'CMakeLists.txt') : cmd = cmake
-DCMAKE_EXPORT_COMPILE_COMMANDS=ON
  elif has(repo, 'Makefile')       : cmd = bear -- make            # 构建拦截[73]
  elif has(repo, '*.ewp|*.uvprojx'): cmd = IdeProjectParser(xml)   # 自研：翻译为
clang 命令行
  ccdb = run(cmd)                                      # compile_commands.json
  for entry in ccdb:                                   # 宏/头文件补全
```

```
    missing = probe_missing_headers(entry)          # 预处理探测
    entry += resolve(missing, sdk_index[target])   # SDK 包索引补全
  image = dockerize(toolchain[target], ccdb)        # 工具链容器固化
  return ccdb, image
```

解决机理：P1 的本质是分析器看到的编译语义与真实构建不一致。本模块以“拦截真实构建优先、解析工程文件兜底”双通道产出编译数据库，并以容器固化，使 L2 层 Clang IR 与各引擎获得与目标构建一致的宏、类型宽度与头文件语义，直接消除 2.2.1/2.2.2/2.2.6 所列的接入失败与解析偏差。

3.3.2 硬件语义与并发建模（对应 P2）

输入/输出：输入为 CMSIS-SVD 芯片描述[74]、链接脚本、启动文件与 RTOS API 调用；输出为寄存器语义库（地址/访问权限/位域）、ISR 并发图（入口、优先级、抢占关系）与 RTOS API 语义标注表。

技术选型：寄存器语义：解析 CMSIS-SVD（XML，覆盖主流 Cortex-M 厂商）生成“地址→外设/寄存器/访问权限”映射，可直接复用开源解析库 cmsis-svd[96]，注入分析器作为 MMIO 对象模型[74]；ISR 入口识别：解析启动文件向量表与链接脚本、匹配中断属性注解三路合一；抢占关系：由 NVIC 优先级与 RTOS 调度配置推导偏序；RTOS 语义为 FreeRTOS/RT-Thread 关键 API（任务创建、队列/信号量、延时、临界区）建立标注表（阻塞点、锁语义、ISR 变体），转换为分析器可消费的模型——静态检查侧扩展 Goblint 式线程模块为“ISR/任务”模块[59]，模型检查侧映射为 ESBMC 并发编码[54]。

```
procedure HwModelExtract(repo, svd):
  regmap = ParseSVD(svd)          # cmsis-svd 解析[96]: 地址 -> (外设, 寄存器, 权限,
位域)[74]
  isrs   = VectorTable(startup) + AttrScan('__attribute__((interrupt))')
  prio   = NVIC_Levels(linker, config)             # 中断优先级偏序
  rtos   = { api: Sem(api) for api in RTOS_APIS }   # 阻塞点/锁/队列语义标注
  annotate(IR, regmap, isrs, prio, rtos)           # 注入统一 IR（L2）
  return regmap, isr_graph, rtos_table
```

解决机理：P2 的根源是语义不可见。将寄存器访问、ISR 抢占与 RTOS 阻塞语义显式注入 IR 后：只读寄存器被写、位域越界、ISR 与任务共享变量无临界区、任务内调用阻塞 API 等缺陷从“不可判定”变为“可规则化检查”；volatile 相关误报因访问对象语义

明确而大幅减少。此外，regmap 与 isr_graph 同时是 3.3.5 动态通道的激励建模与 3.3.6 监测点生成的输入——一次建模，静动三用，这也是与 P2IM 等动态侧外设建模工作（仅服务仿真执行[89]）最本质的区别。

3.3.3 MISRA/GJB/CERT 规则引擎实现（对应 P3）

输入/输出：输入为 L2 统一 IR 与规则集配置；输出为 SARIF 格式的规范告警（含规则编号、等级、证据位置）。

技术选型：规则形式化流水线：规则文本→可判定性分类（句法可判/需类型计算/需数据流/需人工评审）→实现载体。句法类以 Semgrep 模式或 clang-tidy 自定义 check 实现类型计算类（如 MISRA 10.x 整型转换族）需实现 C 标准整型提升模型后在 AST 上计算；数据流类复用 L3 引擎。规则库分三集管理：MISRA C:2012（以 cppcheck misra.py 为基、补齐其未覆盖项[32]）、GJB 8114-2013（124 条自研映射，军事标准文本按授权使用）、CERT C（与 CWE 官方映射[18]）。裁剪策略：按 Mandatory/Required/Advisory 与行业配置文件（汽车/工控）分层启停。

```
# 例：MISRA C:2012 Rule 10.x 族（整型转换）的可判定实现
procedure CheckRule10_3(ast):
  for assign in ast.matches('a = b'):
    tl, tr = UsualArithType(assign.lhs), UsualArithType(assign.rhs)  # C 整型提升模型
    if Rank(tr) > Rank(tl) or Signedness(tr) != Signedness(tl):
      emit_sarif(rule='MISRA C:2012 10.3', level='Required',
                 loc=assign.loc, evidence=[type_of(assign.lhs),
type_of(assign.rhs)])
```

解决机理：P3 的割裂源于规则集与输出体系分离。本模块将规范检查与缺陷检查统一在 L2 IR 与 SARIF 模型上，同一函数的 MISRA 违规与 CWE 告警由 L5 聚合为单一证据链；GJB 规则的自建映射填补 2.2.9 所述开源工具无 GJB/MISRA 支持的空白。

3.3.4 抽象解释与符号执行的嵌入式适配（对应 P4）

输入/输出：输入为带硬件语义标注的 IR 与验证目标（性质/范围）；输出为分级的证明结论或带反例的告警。

技术选型：值域分析以区间域为基底、关键变量升级符号域（参照 Cppcheck value flow 与 IKOS 域设计[32][58]）；循环上界三来源：SVD 中 DMA 长度等硬件约束[74]、工程配置常量、人工标注上限；指针别名采用 Steensgaard/Andersen 分析供给各引擎；状态爆炸控制采用函数摘要式组合（Infer 路线[28]）加大程序分片（CodeSonar 经验[47]），完备性要求高的性质转 ESBMC k-归纳[54]。

解决机理：P4 要求在漏报、误报与成本间给出工程折衷且结论可分级。本模块不追求单一引擎全覆盖，而是按性质类别路由：运行时错误类走健全的抽象解释（可证明）、关键断言走 BMC（界内完备）、广谱缺陷走规则与数据流（低误报）——L5 依此输出“证明/检出/疑似”三级结论，替代单一告警流。

3.3.5 动态分析引擎：模糊测试与仿真执行（对应 P7）

输入/输出：输入为构建复现产物（3.3.1）、硬件语义模型（3.3.2 的 regmap 与 isr_graph）与目标平台约束；输出为真实触发的缺陷证据（崩溃输入 PoC、复现步骤、执行轨迹）与覆盖率报告。

技术选型——执行载体分层：嵌入式 Linux 目标在宿主机以 QEMU 用户态加 ASan/UBSan 插桩直接执行；裸机与 RTOS 固件以 Renode 外设级仿真承载（支持多节点与外设模型，MIT 许可[83]）；单固件镜像的快速模糊测试以 Unicorn CPU 级仿真承载（轻量、启动快[84]）；真机接入（调试口注入激励）列为试点扩展。载体选择原则：保真度要求越高越靠 Renode，吞吐要求越高越靠 Unicorn——两者与宿主机构成三级吞吐-保真梯度。该分层承袭 Firm-AFL 的“系统仿真兼容性 + 用户仿真吞吐”混合思路[88]并将其延伸到裸机/RTOS 目标。

技术选型——harness 自动生成：动态通道的第一道墙是 harness 编写成本（2.2.10[34]）。本模块以 L2 元数据为骨架、LLM 生成初版 harness 与激励模型，编译与执行日志回传驱动修正。该机制在通用生态已被规模化验证：Google OSS-Fuzz 自 2024 年 5 月起以 LLM 为未覆盖项目自动合成 harness[94]，ChatAFL 与 Fuzz4All 分别验证了 LLM 引导协议与多语言 fuzzing[92][93]；本模块的增量在接地信息与执行载体——以

SVD 输入域与 ISR 入口表（3.3.2）替代纯代码上下文，运行于仿真载体服务裸机/RTOS 目标。harness 必须经过“生成→编译→运行→修正”闭环后才投入 fuzzing。

技术选型——MMIO 感知激励建模：纯字节变异难以命中外设交互路径（2.2.10[34]）。将 3.3.2 的寄存器语义库转译为 fuzzing 输入域的语义分片：外设状态空间→输入字节段的映射表，使变异算子感知“该字节写的是 UART 波特率寄存器”而非盲改；配合静态通道给出的可疑路径作为覆盖引导种子。需要诚实定位的是：外设建模本身已有成熟先行工作——P2IM 以运行时观测自动推断外设接口模型[89]、HALucinator 以人工 HAL 模型库替换实现 re-hosting[90]、μAFL 将其推向 MCU 驱动 fuzzing[91]。本模块的差异有两点：其一，采用厂商 SVD 声明式描述直接获得寄存器语义，无需 P2IM 式运行时推断的冷启动开销，也不受 HALucinator 模型库积累的限制；其二，同一份语义模型复用于静态检查器（3.3.3）与运行时监测点（3.3.6）——“一次建模、静动三用”是与上述工作最本质的区别。

技术选型——检测器与 oracle：宿主/嵌入式 Linux 环境启用 ASan（内存错误）与 UBSan（UB，含 standalone 模式供裸机交叉编译使用）；仿真器侧以 Unicorn/Renode 的内存访问钩子检查越界与未映射访问、以看门狗超时捕获死循环与活锁；运行时断言类 oracle 由 3.3.6 监测点供给。

```
procedure DynamicEngine(target, hwmodel):
  carrier  = pick(target)                    # QEMU-user / Renode / Unicorn 三级梯度[83][84]
  harness  = LlmGenHarness(L2.meta, hwmodel) # LLM 生成 harness 与激励模型[92-94]
  while not compiles_and_runs(harness):
      harness = LlmFix(harness, build_log)   # 生成-验证闭环
  inputs  = MmioAwareInputs(hwmodel.regmap)  # SVD 语义 -> 输入域分片（对比 P2IM 运行时推断[89]）
  seeds   = FromStaticSuspects(static_pool)  # 静态可疑路径作覆盖引导种子
  crashes = Fuzz(carrier, harness, inputs, engine='AFL++', seeds=seeds)   # [34]
  for c in crashes:
      emit_sarif(level='Detected', evidence=[c.input, c.trace, c.sanitizer])
  return crashes, coverage_report
```

解决机理：P7 的三道墙逐一化解：环境依赖→仿真载体（Renode/Unicorn 使裸机固件无需真机即可执行，承 Firm-AFL/HALucinator 路线[88][90]）；harness 成本→LLM 生成-验

证闭环（承 OSS-Fuzz harness 合成路线[94]）；MMIO 激励→SVD 语义分片（对 P2IM 的声明式替代[89]）。动态发现的结论自带真触发证据，误报率低，直接进入 L5 可证明性分级的“检出（Detected）”级，与静态通道的“证明（Proven）/疑似（Suspected）”形成互补的三级体系。

3.3.6 运行时监控子系统（对应 P7）

输入/输出：输入为监控性质的三类来源（规则与安全性质、静态通道 Suspected 级结论、LLM 归纳的不变量候选）与 3.3.5 的仿真执行会话；输出为运行时事件流（违例事件、时序统计、寄存器访问序列）与回填 SARIF 的确认/否证证据。

技术选型——监测点生成三来源：规则驱动：将 MISRA/CERT 中可运行时化的子集与安全性质编译为断言（E-ACSL 式运行时断言检查[53]）；研判驱动：静态通道的 Suspected 级结论（如 ISR 竞争嫌疑、可疑越界路径）自动生成定向监测点——共享变量访问序列记录、临界区占用检查、缓冲区边界哨兵；模型驱动：LLM 从代码语义归纳不变量候选（循环边界、状态机合法迁移、寄存器访问序列模式），候选必须先经静态引擎核验（可满足性、与已知约束一致性）后才启用，防止错误不变量制造伪告警。

技术选型——观测项与开销分级：观测四类：内存安全事件、时序（执行时长超限、中断延迟）、调度与中断序列异常、寄存器访问与 SVD 权限模型的偏离[74]。开销控制按监控强度三级配置：关键路径仅挂计数器级检查（低开销常开）；完整断言与 trace 记录放离线重放模式；按 ASIL/安全等级决定各级默认开关。

技术选型——静动联动闭环：监控是“静态怀疑、动态裁决”的执行者：静态通道给出 Suspected 告警→动态测试智能体将其转为定向监测点→仿真执行中命中即回填事件轨迹至 SARIF 并升级为 Detected，未命中且覆盖充分则降级排除。这一闭环同时服务误报过滤（P4/P5）：LLM 的误报判定可由监控结果交叉验证。

```
procedure RuntimeMonitor(sim_session, suspected, properties):
  points  = CompileAssertions(properties)          # E-ACSL 式断言[53]
  points += DirectedProbes(suspected)              # 静态 Suspected -> 定向监测
  invars  = StaticVerify(LlmInvariants(code))      # LLM 不变量候选先经静态核验
```

```
  for event in Simulate(sim_session, points + invars):   # Renode/Unicorn 执行
[83][84]
      if violates(event):
          backfill_sarif(event.loc, event.trace)  # 证据回填
          regrade(event.alert)                    # Suspected -> Detected / 排除
  return event_stream, timing_stats
```

解决机理：P7 的观测开销以三级强度配置化解；更重要的是，运行时监控为静态分析最不可判定的并发时序类缺陷提供了裁决手段——ISR 竞争在仿真中表现为可观测的访问交错事件，Goblint 类线程模型的盲区（2.4.5[59]）由此绕过。

3.3.7 多智能体协同框架（对应 P5、P7）

采纳边界先行：本框架将 LLM 置于协同中枢（任务分解、语义研判、测试与监测点生成、修复建议），而非最终裁决者——严格基准下 LLM 直接判定接近随机[65]，而“LLM 生成、工具验证”的闭环已被 GPTScan[39]、固件修补工作[77]与 FirmAgent[85]实证有效；多智能体的组织形式亦须以数据说话：MAST 对多智能体失败模式的标注[79]与 Agentless 的低成本流水线对照[78]共同表明，自由辩论式协作与无验证的长链路编排是负收益的，本框架以结构化交接与确定性校验规避之，并保留单智能体串行流水线作为消融对照（3.5.3 配置 A6）与降级回退路径。

六类智能体的编成、工具与约束见表 3-2。每个智能体是“LLM 推理 + 专用工具集 + 确定性输出校验器”的组合体：LLM 负责语义决策，工具负责事实与验证，校验器负责格式与证据链完整性。

表 3-2 多智能体编成：职责、工具与验证约束

智能体	职责	可用工具	输出	验证约束
规划智能体	任务分解、引擎路由、结果汇聚判定、成本预算控制	影响集计算、任务队列、预算表	分析计划（结构化 JSON）	路由规则确定性优先，规划不改写技术结论
静态研判智能体	告警分诊、排序、误报过滤、根因解释	SARIF 告警池、RAG 知识库、代码切片	分诊结论+证据链	证据行号程序化核对；低置信转检索或人工
动态测试智能体	harness 与激励模型生成、oracle 设计、覆盖率缺口分析	编译器、AFL++[34]、Renode/Unicorn[83][84]	可运行 harness、PoC、覆盖率报告	harness 须编译运行通过；PoC 须可复现

智能体	职责	可用工具	输出	验证约束
运行时监控智能体	监测点配置、不变量归纳、trace 异常分析	断言编译器[53]、仿真事件流[83]、SVD 模型[74]	监测配置、异常分析报告	不变量先经静态核验方可启用
修复智能体	补丁生成与迭代修补	编译/静态/CBMC 验证[55]、3.3.5 仿真回归	补丁+验证记录	未通过验证闭环的补丁不入库[77]
报告智能体	证据链汇编、CWE/MISRA 映射、合规报告生成	映射表[18][19]、报告模板	检测报告、合规记录	映射表人工审定；数字须溯源到引擎原始输出

协同协议基于 LangGraph 状态机实现[86]，要点有三。其一，结构化交接：智能体间传递的是带 SARIF 引用的状态对象而非自由文本，杜绝上下文丢失与口径漂移。其二，三类控制流：告警分诊采用并行扇出（每条告警独立研判、失败可容忍）；主链路采用顺序流水（构建→静态→动态→监控→聚合）；高危项采用条件升级（interrupt 转人工复核，人工否决即关闭）。其三，评审委员会仅用于高危项：对被判定为误报的高危告警，参照 MARS 的评审结构召集多智能体交叉复核[80]，避免常设委员会的 token 开销[81]。

```
procedure Orchestrate(repo):
  plan  = Planner.Decompose(repo, budget)             # 任务分解/引擎路由/成本预算
  stat  = StaticChannel(plan)                         # 3.3.3/3.3.4 静态通道
  fan   = Parallel([Triage(a) for a in stat.alerts]) # 分诊扇出（3.3.8）
  dyn   = DynamicEngine(plan.targets, hwmodel)        # 动态通道（3.3.5）
  mon   = RuntimeMonitor(dyn.session, fan.suspected, properties)  # 监控（3.3.6）
  fixes = RepairAgent(fan.confirmed + dyn.crashes)    # 补丁 -> 工具验证闭环
  gate  = HumanReview(items where severity >= HIGH)  # 人工复核闸门
  return Aggregate(fan, dyn, mon, fixes, gate)        # L5 聚合（3.3.10）
```

解决机理：P5 的上下文受限由结构化交接与代码切片缓解，幻觉由双层校验（输出校验器+工具验证闭环）抑制；P7 的编排复杂性（三通道产物关联）由规划智能体统一调度。成本风险以三重手段控制：告警分层路由（低危不过 LLM）、分级模型调用（分诊轻量、修复规划用强模型）、单项目 token 预算熔断——多智能体约为单对话 15 倍的 token 乘数[81]与算力归一后增益收窄的实证[82]，决定了多智能体必须以消融数据（A6 对照）证明净增益，而非默认开启。

3.3.8 LLM 研判与修复机制（智能体内部机制，对应 P5）

输入/输出：输入为 L3 产出的 SARIF 告警及其上下文切片；输出为告警排序、误报判定、修复建议，每条结论附证据链并经工具验证。本节内容由 3.3.7 的静态研判智能体与修复智能体执行。

技术选型：（1）上下文裁剪：以告警函数为核心，纳入深度不超过 2 的调用者/被调用者闭包、相关类型与宏定义，按 token 预算截取（回应仓库级上下文受限[65][66]）。（2）检索增强：知识库含 MISRA/CERT 条款解释、CWE 描述[19]、芯片手册片段与历史缺陷案例；采用置信度触发检索（A-RAG 模式）——高置信直接输出、低置信检索后重判该模式据报道在 Toyota ITC 与 Big-Vul 统一数据集上取得 6.7% 至 18% 的 F1 提升[待核实]，其开源实现地址亦待进一步核实。（3）幻觉抑制与证据回链：强制结构化输出（JSON：判定/置信度/证据行号/理由），判定所引用的行号经程序化核对，未通过核对的结论降级为“疑似”；修复建议进入验证闭环——编译诊断、静态复查、CBMC 性质检查与 3.3.5 仿真回归依次通过后才允许采纳[55][77]。（4）告警排序：规则严重度、路径可达性与 LLM 判定融合评分。（5）本地化部署：以 vLLM 部署开源权重模型满足数据不出域要求，分级调用控制成本。

```
procedure LlmTriage(alert):
  ctx  = SliceContext(alert, depth=2, budget=4k)      # 函数+调用闭包+类型/宏
  p1   = LlmJudge(alert, ctx)                          # 首判：结构化 JSON
  if p1.confidence < THRESH:                           # 置信度触发检索（A-RAG 模
式）
      kb   = Retrieve('MISRA|CWE|手册', alert.rule)    # RAG：条款/手册/案例
      p1   = LlmJudge(alert, ctx + kb)                 # 携带知识重判
  if not VerifyLines(p1.evidence, ctx):                # 证据回链：行号核对
      p1.verdict = 'SUSPECTED'                         # 未通过核对即降级
  if p1.verdict == 'FIX_SUGGESTED':
      p1.patch_v = ToolLoopVerify(p1.patch)            # 编译+静态+CBMC+仿真验证闭
环[55][77]
  return p1
```

3.3.9 增量分析与大规模工程性能优化

输入/输出：输入为版本差异（diff）与分析基线；输出为受影响分析集与增量告警流。

技术选型：影响集计算：修改文件→头文件包含闭包→调用图反向闭包→受影响编译单元与函数集合；告警基线管理参照 CodeChecker 的存储/去重/抑制机制[30][53]：新旧告

警差分，仅新增告警进入研判；执行层以任务队列并行调度各引擎，深度引擎（BMC/抽象解释）与动态通道（fuzzing/仿真）仅对影响集内函数与可达入口运行；动态通道的harness 与激励模型按目标平台缓存复用，避免重复生成。

解决机理：直接回应 2.6 节原因（8）：以影响集将静态深度分析与动态 fuzzing 的触发范围从全库收缩到变更闭包，使深度能力可以嵌入每日集成，而非一次性离线行为。

3.3.10 结果聚合、可证明性输出与报告生成（呼应 P3、P4）

输入/输出：输入为 L3 双通道与 L4 全部结论；输出为聚合后的统一报告：检测报告、MISRA/GJB 合规记录与证据链存档。

技术选型：聚合：以 SARIF 为唯一交换格式做跨引擎去重与合并（同位置同类别合并、保留各引擎独立结论）；映射：维护 CWE↔MISRA↔GJB 双向映射表（CERT C 与 CWE 有官方映射[18]），使每条告警同时携带规范与漏洞双重口径；可证明性分级：Proven（抽象解释/模型检查给出健全证明）、Detected（动态真触发，附 PoC 与轨迹；或引擎检出且研判/人工确认）、Suspected（LLM 研判或未确证告警）——动态通道的引入使 Detected 级证据从“推断”升级为“可复现事实”；报告：按 MISRA Compliance 框架输出合规记录（每规则：合规/偏离/豁免与证据），满足 1.1 节标准对证据链的可追溯要求。

3.4 数据集

3.4.1 合成/标准数据集

Juliet C/C++ 测试集（NIST SARD）：1.2 版含 61,387 个 C/C++ 用例、覆盖 118 种 CWE，每个用例含 good/bad 配对，现行 1.3 版[67]。用途：规则引擎与工具检出率的覆盖广度评测，亦可为动态通道提供带标准答案的触发目标。局限：人工合成的教科书式缺陷，good/bad 配对使模型可学到结构捷径，与真实 CVE 分布严重不符[65]。RERS Challenge：自动生成的反应式系统正确性基准（2011 年起）[68]；SV-COMP Benchmark Archive：

2025 届 34,700 个验证任务，含 witness 校验闭环[69]。用途：形式化引擎的可达性/内存安全回归评测。

3.4.2 真实世界数据集

CVEfixes（PROMISE 2021）：从 NVD 自动抓取 CVE 并关联开源仓库修复提交，构建关系数据库；其依赖 NVD 映射质量，混杂提交（tangled commits）引入标签噪声[70]。Big-Vul（MSR 2020）：3,754 个漏洞函数、91 种漏洞类型、348 个项目[71]。ReposVul（ICSE 2024 Industry）：首个仓库级数据集，6,134 个 CVE、236 种 CWE、1,491 个项目，保留仓库上下文并过滤混杂补丁[72]。PrimeVul（ICSE 2025）：严格去重、按时间划分的函数级基准（约 23 万函数、6,968 个漏洞函数），揭示既有基准高估[65]。本系统采用上述数据集时遵循 PrimeVul 的教训：严格去重、按时间划分、报告配对评估结果。

3.4.3 自建嵌入式数据集（创新点，单独详写）

现有任何公开数据集均不携带嵌入式硬件上下文（ISR/MMIO/RTOS）标注，这既是 P6 的直接表现，也是本系统数据侧的增量空间。设计如下。

（1）来源仓库。按行业相关性与缺陷密度选取十个开源仓库：RT-Thread、Zephyr、FreeRTOS-Kernel、lwIP、U-Boot、OpenWrt、PX4、ArduPilot、ARM mbed OS 及 Linux 内核驱动子系统。选择理由：覆盖 RTOS 内核、网络协议栈、引导程序、无人机与工控固件等典型嵌入式形态，且全部开源可追溯。

（2）采集流程（六步）。第一步，从 NVD/GitHub Security Advisory 与各仓库官方安全通告抓取 CVE/GHSA 条目；第二步，以 CVE 编号与安全关键词检索 git log，人工确认修复提交（fix commit）；第三步，提取漏洞函数及修复后版本切片（含 diff 与调用上下文）；第四步，双标注加仲裁：每样本由两名标注者独立标注，分歧仲裁，标注字段见下；第五步，清洗与去重（见（4））；第六步，按时间划分入库。

（3）标注规范。每样本标注：CWE 编号（主）、缺陷类型（内存/并发/数值/资源/其他，对齐 Toyota ITC 九类口径）、严重等级（对齐 GB/T 30279-2020 分级[6]）、嵌入式

上下文标记（是否涉及 ISR、MMIO、RTOS API、交叉编译相关代码，二值或多值）、修复摘要；二期增补动态标注字段——触发输入（PoC）与监测命中记录，凡动态通道可复现的样本标记为“已触发”，用于双通道互补率评测。

（4）清洗与去重。近重复删除：以函数规范化文本（去注释/空白）做指纹聚类，仅保留代表样本（参照 PrimeVul 方法[65]）；编译过滤：无法在目标平台配置下解析的样本剔除；混杂提交：修复 diff 中与安全无关的改动（重构/文档）通过提交切片对齐剔除。

（5）规模预估与划分。一期目标：800 至 1,200 个漏洞函数，配对正常函数 8,000 至 12,000 个（约 1:10）；按修复时间划分训练/验证/测试集（约 6:2:2），杜绝时间泄漏。

（6）类别不平衡处理。配对采样（每漏洞样本配对同类正常样本）、损失加权与按 CWE 分层抽样三管齐下，评测时同时报告宏平均与按类 F1。

（7）许可证合规。逐仓库登记许可（GPL 系、Apache-2.0、BSD 系）；数据集仅发布函数级切片特征与标注（不整仓再分发），研究用途声明与源仓库引用随数据集发布，规避许可证传染。

表 3-3 自建嵌入式漏洞数据集设计

设计维度	方案
来源仓库	RT-Thread、Zephyr、FreeRTOS-Kernel、lwIP、U-Boot、OpenWrt、PX4、ArduPilot、ARM mbed OS、Linux 驱动子系统
采集流程	CVE/GHSA 抓取 → git log 定位修复提交 → 漏洞/修复函数切片 → 双标注+仲裁 → 清洗去重 → 时间划分
标注规范	CWE（主）、缺陷类型（对齐 Toyota ITC 九类）、严重等级（对齐 GB/T 30279-2020）、嵌入式上下文标记（ISR/MMIO/RTOS/交叉编译）；二期增补 PoC 与监测命中记录
规模预估	一期 800—1,200 个漏洞函数，配对正常函数 8,000—12,000（约 1:10）
清洗与去重	函数指纹近重复删除（PrimeVul 方法）、编译过滤、混杂提交切片对齐
划分方式	按修复时间划分训练/验证/测试（约 6:2:2），杜绝时间泄漏
类别不平衡	配对采样、损失加权、按 CWE 分层抽样；报告宏平均与按类 F1
许可证合规	逐仓库登记许可；仅发布切片特征与标注，不整仓再分发；研究用途声明

3.4.4 数据集使用策略

评测组合：Toyota ITC 与 Juliet 用于与既有文献可比的基准评测；自建数据集用于嵌入式特有能力的评测与消融；SV-COMP/RERS 用于形式化引擎回归。训练（若涉及微调，见 3.6 扩展项）仅使用训练划分，测试划分在系统冻结前不解封。

3.5 评测方案与指标

3.5.1 对比基线

从第二章选区四个可获取工具作为基线：Cppcheck（开源预筛选代表，含 MISRA 插件）[32]；Clang Static Analyzer（经 CodeChecker 封装，路径敏感代表）[30]；CodeQL（查询式 SAST 代表）[26]；ESBMC（形式化代表）[54]。可选项：TscanCode[33]（注意其已停更）。基线与本系统在同一构建复现产物（3.3.1）上运行，保证输入等价。动态侧另以固件 fuzzing 研究工具（P2IM/HALucinator/μAFL[89][90][91]，视环境可得性）作为动态通道的参照点，单独报告其触发数与覆盖率供横向参考。

3.5.2 指标定义

（1）精确率（Precision）、召回率（Recall）、F1：函数级与告警级双口径分别报告；（2）误报审查成本：误报数乘以平均单条审查时间，参照微软对开发者告警处置行为的实证设定观测口径[75]；（3）每千行分析耗时（全量与增量两种模式）；（4）CWE/MISRA 覆盖率：可检出规则数占规则集总数的比例；（5）可证明性比例：Proven 与 Detected 级结论占全部结论的比例；（6）动态通道贡献率：动态新发现且静态未报的缺陷占全部确认缺陷的比例——该指标直接度量双通道互补性，是判断动态通道投入产出比的核心依据。

3.5.3 消融实验设计

为验证 LLM 多智能体、硬件语义建模、动态通道等各模块的独立贡献，设计七配置消融（表 3-4），在自建数据集与 Toyota ITC 上运行；每配置重复三次取均值（LLM 采样温度固定并记录种子）。

表 3-4 消融实验设计

配置	说明	验证的问题
A0（完整系统）	五层全开：双通道+硬件语义+多智能体+验证闭环	整体效果（对照基线）
A1（去 LLM 协同层）	移除 L4，仅保留双通道工具流水线	LLM 协同的净贡献（对应 P5）
A2（去硬件语义）	移除 3.3.2 的 ISR/MMIO/RTOS 模型	硬件语义建模的净贡献（对应 P2）
A3（去 RAG）	研判智能体不接知识检索	领域知识注入的净贡献
A4（去验证闭环）	LLM 修复建议不经验证直接采纳	工具验证回路的净贡献（对应 P5 幻觉抑制）
A5（去动态通道）	L3 仅保留静态通道，无 fuzzing/监控	双通道互补的净贡献（对应 P7）
A6（单智能体串行）	多智能体退化为单 LLM 顺序处理	多智能体架构的净增益（对照 Agentless 证据[78]）

3.5.4 预期结果表（留空待填）

表 3-5 为预期结果记录模板，实验完成后以实测数据填入；除表列指标外，同步记录各配置的失败案例归因（构建失败/引擎超时/LLM 幻觉/仿真不可达等），作为迭代依据。

表 3-5 评测结果记录模板（待填）

系统配置	精确率	召回率	F1	误报审查成本	千行耗时	可证明性比例	动态贡献率
Cppcheck（基线）	待填	待填	待填	待填	待填	—	—
Clang SA（基线）	待填	待填	待填	待填	待填	—	—
CodeQL（基线）	待填	待填	待填	待填	待填	—	—
ESBMC（基线）	待填	待填	待填	待填	待填	待填	—
A0 完整系统	待填	待填	待填	待填	待填	待填	待填
A1 去 LLM 协同层	待填	待填	待填	待填	待填	待填	待填
A2 去硬件语义	待填	待填	待填	待填	待填	待填	待填
A3 去 RAG	待填	待填	待填	待填	待填	待填	待填
A4 去验证闭环	待填	待填	待填	待填	待填	待填	待填
A5 去动态通道	待填	待填	待填	待填	待填	待填	—
A6 单智能体串行	待填	待填	待填	待填	待填	待填	待填

3.6 实施路线与阶段计划

计划以季度为粒度，共五个阶段（2026 年第四季度至 2027 年第四季度），每阶段设可验收里程碑；静态与动态通道并行推进但动态通道晚一个季度启动（依赖 3.3.1/3.3.2 的产出），多智能体协同层在静态闭环之后引入以控制风险。

表 3-6 分季度实施里程碑

时间	阶段	关键内容	里程碑与验收标准
2026 Q4	一、构建复现与规则引擎 MVP	3.3.1 构建复现工具链；MISRA C:2012 子集规则引擎；Juliet 接入	选定目标仓库零人工干预完成接入与全量扫描（P1 闭环）
2027 Q1	二、静态深度引擎与动态通道 MVP	3.3.2 SVD/ISR/RTOS 建模；抽象解释与符号执行适配；Renode/Unicorn 仿真执行与 UBSan 接入；harness 生成-验证闭环试跑	P2/P4 机制落地；Toyota ITC 首轮评测完成；动态通道对至少 1 个目标固件产出触发证据
2027 Q2	三、多智能体协同层 v1 与数据集	规划/研判/修复/动态测试智能体与验证闭环（3.3.7/3.3.8）；MMIO 感知激励建模；自建数据集 v1（不低于 800 个漏洞函数）	消融 A0—A4 完成（P5 闭环）；F1 相较基线提升 20% 目标首次检验
2027 Q3	四、运行时监控与全量评测	运行时监控子系统（3.3.6）；消融 A5/A6；四基线对比评测；3.3.10 合规报告	评测报告与论文/技术报告成稿（P3/P6/P7 闭环）；动态贡献率与双通道互补率实测
2027 Q4	五、试点部署与迭代	汽车电子或工控合作项目试点；真机接入与仿真保真度校准；反馈驱动迭代	试点反馈报告与二期计划

风险与回退：若阶段三评测显示 LLM 协同增益不显著（A1 与 A0 差距小于预设阈值），回退方案为强化规则库与人工复核工作流，五层架构骨架不变——Agentless 的证据表明简单流水线不可轻易放弃[78]，A6 配置即为此保留的降级形态；若动态通道仿真保真度不足导致假阴性，以真机抽测校准仿真模型并将动态结论降级为辅助证据；若自建数据集规模不达预期，评测主基准回退至 Toyota ITC 加 Juliet 组合并明确声明局限；闭源 IDE 工程解析（.ewp/.uvprojx）若受阻，以导入 CMake 化的等价工程兜底。所有阶段产出（工具、数据集标注规范、评测脚本）随阶段验收同步归档，保证可复现。

参考文献

以下文献与官方链接均于 2026 年 10 月经联网核实；未附链接者为未核实到稳定官方链接的条目（正文中已相应标注），正式引用前建议二次确认。正文引用处标注为 [N]。

[1] 全国人民代表大会常务委员会. 中华人民共和国网络安全法（2016 年通过，2017 年 6 月 1 日施行；2025 年 10 月修正，2026 年 1 月 1 日施行）. https://www.cac.gov.cn

[2] 国务院. 关键信息基础设施安全保护条例（国务院令第 745 号，2021 年 9 月 1 日施行）. https://www.gov.cn

[3] 全国人民代表大会常务委员会. 中华人民共和国数据安全法（2021 年 9 月 1 日施行）. https://www.cac.gov.cn/2021-06/11/c_1624994566919140.htm

[4] GB/T 22239-2019 信息安全技术 网络安全等级保护基本要求（2019-05-10 发布，2019-12-01 实施）. https://openstd.samr.gov.cn

[5] GB/T 38674-2020 信息安全技术 应用软件安全编程指南（2020-04-28 发布，2020-11-01 实施）. https://openstd.samr.gov.cn/bzgk/std/newGbInfo?hcno=910B7DBAEB214FE027F163E361960208

[6] GB/T 30279-2020 信息安全技术 网络安全漏洞分类分级指南（2021-06-01 实施）. https://openstd.samr.gov.cn

[7] GB/T 34590（所有部分）-2022 道路车辆 功能安全（对应 ISO 26262:2018，2023-07-01 实施）. https://openstd.samr.gov.cn

[8] GB/T 20438（所有部分）-2017 电气/电子/可编程电子安全相关系统的功能安全（对应 IEC 61508:2010）. https://openstd.samr.gov.cn

[9] 工业和信息化部. “十四五”软件和信息技术服务业发展规划（工信部规〔2021〕180 号，2021-11-15 印发）. https://www.ncsti.gov.cn/zcfg/zcwj/202111/t20211130_52282.html

[10] 国务院办公厅. 新能源汽车产业发展规划（2021—2035 年）（国办发〔2020〕39 号，2020-11-02 发布）. https://www.mee.gov.cn/zcwj/gwywj/202011/t20201103_806156.shtml

[11] 工业互联网专项工作组. 工业互联网创新发展行动计划（2021—2023 年）（工信部信管〔2020〕197 号，2020-12 印发）. http://www.scio.gov.cn/xwfb/gwyxwbgsxwfbh/wqfbh_2284/2021n_2711/2021n03y08r/wjxgzc_2910/202208/t20220808_310230.html

[12] 中央网络安全和信息化委员会. “十四五”国家信息化规划（2021-12-27 发布）. http://www.cac.gov.cn/2021-12/27/c_1642205314518676.htm

[13] ISO/SAE 21434:2021 Road vehicles — Cybersecurity engineering（2021-08-31 发布）. https://www.iso.org

[14] UNECE Regulation No. 155 — Uniform provisions for the approval of vehicles with regards to cyber security（2021-01 生效，2022-07 起新车型强制）. https://unece.org/sites/default/files/2023-02/R155e%20%282%29.pdf

[15] Regulation (EU) 2024/2847（Cyber Resilience Act，2024-12-10 生效；漏洞报告义务自 2026-09-11、主要义务自 2027-12-11 起适用）. https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act

[16] EN 50716:2023 Railway applications — Requirements for software development（2023-11 发布，取代 EN 50128:2011+A2:2020）. https://standards.iteh.ai/catalog/standards/clc/32d1d924-9bfd-477e-adf4-de6e3358be77/en-50716-2023

[17] MISRA. MISRA C:2012（2013-03 第三版，AMD1 2016 / AMD2 2020 / AMD3 2022 / AMD4 2023）；MISRA C:2023；MISRA C:2025（2025-03）. https://misra.org.uk/publications/

[18] SEI CERT C Coding Standard（CMU Software Engineering Institute，2016 发行版，在线版持续更新）. https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard/

[19] MITRE. 2025 CWE Top 25 Most Dangerous Software Weaknesses（基于约 39,080 条 CVE）. https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

[20] Industrial Cyber. SynSaber 对 CISA ICS advisories 的分析：2022 年揭示 1,342 个 ICS CVE，2020—2022 年增长约 144%. https://industrialcyber.co/news/synsaber-notes-144-rise-in-cves-reported-as-ics-advisories-from-2020-to-2022

[21] Microsoft Security Signals（2021-03）：80% 的受访企业过去两年内遭遇过至少一次固件攻击. https://www.microsoft.com/en-us/security/blog/2021/03/30/new-security-signals-study-shows-firmware-attacks-on-the-rise-heres-how-microsoft-is-working-to-help-eliminate-this-entire-class-of-threats

[22] Upstream Security. Global Automotive Cybersecurity Report 2025（2024 年公开报道汽车网络安全事件 409 起，2023 年为 295 起）. https://upstream.auto

[23] SDV Insider（2025-02）：2024 年美国车辆召回约 2,890 万辆，其中约 46% 涉及软件问题（2023 年为 14.2%；口径为涉及软件的召回，与 NHTSA 官方口径存在差异）. https://www.envorso.com

[24] Krasner H. The Cost of Poor Software Quality in the US: A 2022 Report. CISQ（美国软件劣质成本约 2.41 万亿美元）. https://www.it-cisq.org

[25] Synopsys（现 Black Duck）. Coverity 与 Coverity Scan（2002 年创立，Scan 云服务 2007 年上线，覆盖 3,900+ 开源项目）. https://scan.coverity.com

[26] GitHub. CodeQL 与 QL 查询语言（Semmle 2006 年创立，GitHub 2019 年收购，2019-11 免费开放研究与开源使用）. https://codeql.github.com

[27] r2c. Semgrep（前身 Facebook 内部 sgrep，2020 年改名发布；OSS 引擎 LGPL-2.1）. https://github.com/semgrep/semgrep

[28] Calcagno C, Distefano D, et al. Scaling static analyses at Facebook. Communications of the ACM, 58(8), 2015；Meta Infer（2015-06 开源，MIT）. https://fbinfer.com

[29] SonarSource. SonarQube（2008 年，瑞士日内瓦）. https://www.sonarsource.com

[30] LLVM 项目. Clang Static Analyzer（路径敏感符号执行 + 可插拔 checker 架构，Apache-2.0）. https://clang.llvm.org/docs/ClangStaticAnalyzer.html

[31] PVS-Studio（2008 年，俄罗斯图拉；1,000+ 诊断规则；曾对 ArduPilot 等开源项目出具分析报告）. https://pvs-studio.com

[32] Marjamäki D. Cppcheck（2009-03 仓库创建，GPL-3.0，支持 --platform 嵌入式平台参数）. https://github.com/danmar/cppcheck

[33] 腾讯. TscanCode（2015-03 仓库创建，覆盖 C/C++、C#、Lua；2023 年底后停止更新）. https://github.com/Tencent/TscanCode

[34] Fioraldi A, Maier D, Eißfeldt H, Heuse M. AFL++: Combining Incremental Steps of Fuzzing Research. USENIX WOOT 2020. https://github.com/AFLplusplus/AFLplusplus

[35] Google. OSS-Fuzz（2016-12 上线；官方 README 口径累计发现 13,000+ 安全漏洞、50,000+ bugs）. https://google.github.io/oss-fuzz

[36] Feng Z, Guo D, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. EMNLP 2020. https://aclanthology.org/2020.emnlp-main.207/

[37] Guo D, Lu S, et al. GraphCodeBERT: Pre-training Code Representations with Data Flow. ICLR 2021. https://openreview.net/forum?id=jLoC4ez43PZ

[38] Guo D, et al. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. AAAI 2022. https://arxiv.org/abs/2203.03850

[39] Sun Y, Wu D, et al. GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis. ICSE 2024. https://arxiv.org/abs/2308.03314

[40] Sun Y, Wu D, et al. LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning（2024，arXiv 技术报告）. https://arxiv.org/abs/2401.16185

[41] Engler D, Chelf B, Chou A, Hallem S. Checking System Rules Using System-Specific, Programmer-Written Compiler Extensions（Metal）. PLDI 2001；Bugs as Deviant Behavior. SOSP 2001. https://web.stanford.edu/~engler/deviant-sosp-01.pdf

[42] Das M, Lerner S, Seigle M. ESP: Path-Sensitive Program Verification in Polynomial Time. PLDI 2002（ACM Digital Library 可检索）.

[43] Chen H, Wagner D, Dean D. MOPS: An Infrastructure for Examining Security Properties of Software. ACM CCS 2002. https://dl.acm.org/doi/10.1145/586110.586142

[44] Xie Y, Aiken A. Scalable Error Detection Using Boolean Satisfiability（Saturn）. POPL 2005. http://saturn.stanford.edu

[45] AbsInt. Astrée（Cousot 抽象解释谱系，2001—2003 研制，2009 年由 AbsInt 商用；2003 年对 Airbus A340 主飞控软件约 40 万行 C 完成零误报运行时错误验证，后扩展至 A380 电传飞控）. https://www.absint.com/astree/index.htm

[46] MathWorks. Polyspace Bug Finder / Code Prover（前身 PolySpace Technologies 1999 年创立，MathWorks 2007-04 收购）. https://www.mathworks.com/products/polyspace.html

[47] GrammaTech（现 CodeSecure/AdaCore 体系）. CodeSonar（2003 年首发；NASA 2008 年将其引入 Space Network 地面段分析）. https://www.grammatech.com

[48] Perforce. Klocwork（2001 年自 Nortel Networks 分立，2019 年随 Rogue Wave 并入 Perforce）. https://www.perforce.com/products/klocwork

[49] Perforce. Helix QAC（原英国 Programming Research 的 QA·C/QA·C++，1986 年起；2018-05 被 Perforce 收购）. https://www.perforce.com/products/helix-qac

[50] LDRA（Liverpool Data Research Associates，1975 年成立，英国利物浦；TBrun/TBsafe/LDRAcover）. https://ldra.com

[51] Parasoft. C/C++test（公司 1987 年创立，产品 1999 年推出）. https://www.parasoft.com

[52] Vector. VectorCAST（Vector Software 1990 年创立，2017-01 并入 Vector Informatik；提供 DO-178B/C/DO-330 工具鉴定包）. https://www.vector.com/int/en/products/vectorcast/

[53] Baudin P, et al. The Frama-C Software Analysis Platform. 2021；CEA List + Inria，2008 年首次公开发布；E-ACSL 运行时断言插件；Airbus/Dassault 曾以 ACSL+Frama-C 开展 DO-178C 低层需求验证实验（Dordowsky et al., 2015）. https://frama-c.com

[54] Cordeiro L, Fischer B, et al. ESBMC: A Bounded Model Checker for ANSI-C and SystemC. CAV 2011（v5.0 起改用 Clang 前端）. https://github.com/esbmc/esbmc

[55] Clarke E, Kroening D, Lerda F. A Tool for Checking ANSI-C Programs（CBMC）. TACAS 2004；现由 AWS 自动推理团队主导维护. https://www.cprover.org/cbmc/

[56] Cadar C, Dunbar D, Engler D. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. OSDI 2008. https://www.usenix.org/event/osdi08/tech/full_papers/cadar/cadar.pdf

[57] Beyer D, Keremoglu M E. CPAchecker: A Tool for Configurable Software Verification. CAV 2011（SV-COMP 2012 首届总冠军，多年多赛道夺冠）. https://cpachecker.sosy-lab.org

[58] Brat G, Navas J A, Shi N, Venet A. IKOS: A Framework for Static Analysis based on Abstract Interpretation（NASA Ames，2015）. https://github.com/NASA-SW-VnV/ikos

[59] Vojdani V, Vene V. Goblint: Path-sensitive Data Race Analysis. SPLST 2007（现由 TU Munich 主导；SV-COMP 数据竞争赛道 2021/2023/2024 得分最高）. https://goblint.in.tum.de

[60] Beyer D. SV-COMP: International Competition on Software Verification（TACAS 工具竞赛，2012 年首届；2026 年为第 15 届，74 个验证工具 + 20 个校验器）. https://sv-comp.sosy-lab.org

[61] Li Z, Zou D, Xu S, et al. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. NDSS 2018. https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_03A-2_Li_paper.pdf

[62] Zhou Y, Liu S, Siow J, Du X, Liu Y. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. NeurIPS 2019. https://arxiv.org/abs/1908.09411

[63] Li Z, Zou D, Xu S, et al. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. IEEE TDSC（2021 年在线，2022 年正式刊出）. https://github.com/SySeVR/SySeVR

[64] Zheng Y, Pujar S, Lewis B, et al. D2A: A Dataset Built for AI-Based Vulnerability Detection Methods Using Differential Analysis. ICSE-SEIP 2021. https://arxiv.org/abs/2102.07995

[65] Ding Y, Fu Y, et al. Vulnerability Detection with Code Language Models: How Far Are We?（PrimeVul）. ICSE 2025（arXiv 2024-03）. https://arxiv.org/abs/2403.18624

[66] Gao Z, Wang H, et al. How Far Have We Gone in Vulnerability Detection Using Large Language Models（VulBench）. NAACL 2024. https://arxiv.org/abs/2311.12420

[67] NIST. Juliet C/C++ Test Suite（SARD；1.2 版含 61,387 个 C/C++ 用例、覆盖 118 种 CWE；现行 1.3 版）. https://samate.nist.gov

[68] Howar F, et al. The RERS Challenge（2011 年发起，绑定 ISoLA；反应式系统正确性开放基准）. https://rers-challenge.org

[69] SOSY Lab. SV-COMP Benchmark Archive（2025 届 34,700 个验证任务；witness 校验闭环评分）. https://gitlab.com/sosy-lab/benchmarking/sv-benchmarks

[70] Bhandari G, Naseer A, Moonen L. CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software. PROMISE 2021. https://github.com/secureIT-project/CVEfixes

[71] Fan J, Li Y, Wang S, Nguyen T N. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries（Big-Vul）. MSR 2020（3,754 个漏洞函数，91 种漏洞类型，348 个项目）.

[72] Wang X, et al. ReposVul: A Repository-Level High-Quality Vulnerability Dataset. ICSE 2024 Industry Track（6,134 个 CVE，236 种 CWE，1,491 个项目）. https://arxiv.org/abs/2401.13169

[73] rizsotto. Bear（Build EAR）：生成 compile_commands.json 的构建拦截工具. https://github.com/rizsotto/Bear

[74] ARM. CMSIS-SVD：Cortex-M 微控制器外设寄存器描述标准（XML）. https://github.com/ARM-software/CMSIS-SVD

[75] Habib A, et al. How Do Developers Act on Static Analysis Alerts? An Empirical Study of Microsoft's C, C++ and C# Codebases. FSE 2019.

[76] Risse N, Böhme M. Top Score on the Wrong Exam: 独立重评测研究（揭示漏洞检测基准的重复与泄漏导致性能高估）. 2022.

[77] Abtahi S M, Azim A. Securing LLM-Generated Embedded Firmware through AI Agent-Driven Validation and Patching（GPT-4 + FreeRTOS/QEMU + CWE 分类迭代修补，报告 92.4% 漏洞修复率）. arXiv:2509.09970, 2025-09. https://arxiv.org/abs/2509.09970

[78] Xia C S, et al. Agentless: Demystifying LLM-based Software Engineering Agents（简单结构化流水线以约 1/10 成本取得竞争力的对照证据）. arXiv:2407.01489, 2024. https://arxiv.org/abs/2407.01489

[79] Cemri M, et al. Why Do Multi-Agent LLM Systems Fail?（MAST：多智能体系统失败模式分类学）. arXiv:2503.13657, 2025. https://arxiv.org/abs/2503.13657

[80] MARS: Multi-Agent Review System（评审委员会式多智能体协同审查）. arXiv:2509.20357, 2025. https://arxiv.org/abs/2509.20357

[81] Anthropic. How We Built Our Multi-Agent Research System（多智能体 token 消耗约为单对话 15 倍的工程数据）. 2025-06. https://www.anthropic.com/engineering

[82] Single-Agent LLMs Outperform Multi-Agent Systems（测试算力归一后多智能体增益收窄的实证）. arXiv:2604.02460, 2026. https://arxiv.org/abs/2604.02460

[83] Antmicro. Renode：开源 MCU/SoC 外设级仿真框架（MIT，支持 Robot Framework 测试集成与多节点仿真）. https://github.com/renode/renode

[84] Unicorn Engine：基于 QEMU 的轻量多架构 CPU 仿真框架（GPL-2.0）. https://github.com/unicorn-engine/unicorn

[85] FirmAgent: Leveraging Fuzzing to Assist LLM Agents with IoT Firmware Vulnerability Discovery（以模糊测试证据引导 LLM 智能体，NDSS Symposium）. https://www.ndss-symposium.org

[86] LangChain. LangGraph：面向智能体编排的开源状态机框架（MIT）. https://github.com/langchain-ai/langgraph

[87] Comparative Analysis of LLMs for MISRA C++ Compliance（以 PC-lint Plus 为裁判的实测：无一模型能直接生成完全合规代码）. arXiv:2506.23535, 2025. https://arxiv.org/abs/2506.23535

[88] Zheng Y, Yin H, Davanian A, et al. FIRM-AFL: High-Throughput Greybox Fuzzing of IoT Firmware via Augmented Process Emulation. USENIX Security 2019（约 489 引用；代码 zyw-200/FirmAFL）. https://www.usenix.org/conference/usenixsecurity19/presentation/zheng

[89] Feng B, et al. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. USENIX Security 2020（外设接口自动推断建模；代码 RiS3-Lab/p2im）. https://www.usenix.org/conference/usenixsecurity20/presentation/feng

[90] Clements A, et al. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. USENIX Security 2020（HAL 函数替换实现无硬件固件仿真）. https://www.usenix.org/conference/usenixsecurity20/presentation/clements

[91] µAFL: Hardware-Assisted Fuzzing for Microcontroller Units. ICSE 2022（面向 MCU 外设驱动代码的模糊测试，承 P2IM/HALucinator 思路）. https://ittc.ku.edu/~fli/papers/2022_icse_uafl.pdf

[92] Meng R, et al. Large Language Model guided Protocol Fuzzing（ChatAFL）. NDSS 2024. https://www.ndss-symposium.org/ndss-paper/large-language-model-guided-protocol-fuzzing

[93] Xia C S, et al. Fuzz4All: Universal Fuzzing with LLMs. ICSE 2024. https://lingming.cs.illinois.edu/publications/icse2024a.pdf

[94] Google. Introducing LLM-based harness synthesis for unfuzzed projects（OSS-Fuzz 官方博客，2024-05 上线并规模化）. https://blog.oss-fuzz.com/posts/introducing-llm-based-harness-synthesis-for-unfuzzed-projects

[95] Rivelli G, et al. Karonte: Detecting Insecure Multi-binary Interactions in Embedded Firmware. IEEE S&P 2020（代码 ucsb-seclab/karonte）. https://github.com/ucsb-seclab/karonte

[96] cmsis-svd：CMSIS-SVD 文件的 Python 解析库（供基于 SVD 的工具链开发复用）. https://github.com/cmsis-svd/cmsis-svd

[97] atillab1. embedded-ai-workspace：面向嵌入式固件的三阶段 Claude 工具链（MISRA Skill + Datasheet MCP + HIL 串口流水线；STM32F4-DISCOVERY 端到端验证）. https://github.com/atillab1/embedded-ai-workspace

[98] 阿里巴巴. open-code-review（AI 代码审查工具，确定性流水线 + LLM 智能体混合架构，Apache-2.0）. https://github.com/alibaba/open-code-review